Google

SWE Coding Questions - 31 to 90 days ago

33 questions candidates reported | by source site, each Hard, then Medium, then Easy

Collected 2026-10-10 | one optimal answer each, brute force as pseudo-code, follow-ups

3	29	1
HARD	MEDIUM	EASY

Optimal answers: 7 from the interview site, 7 from your app, 19 written here.
★ = top 75 Medium/Hard or top 10 Easy by Google frequency (6 in this PDF).
Study tiers by Google frequency on LeetCode: TYPE 12, TALK 13, READ 8 (in this PDF).
5 design-style questions are mixed in under their topics, last in each topic and marked DESIGN-STYLE.

Order: source site, recency (last 30 days, 90 days, ...), Hard, Medium, Easy, then strictly newest to oldest

- **1Point3Acres (22): Hard 0, Medium 22, Easy 0**
- **PracHub (7): Hard 2, Medium 4, Easy 1**
- **Exponent (4): Hard 1, Medium 3, Easy 0**

Date pills (colour = how recent): LAST 30 DAYS | LAST 90 DAYS | LAST 6 MONTHS

1Point3Acres: the date the question was last asked. PracHub: the day it was posted there. Exponent: ~ means an estimate from 'N months ago' (it is the last time any company was asked). HackTheRounds shows no per-question dates.

Topics (a tag on every dated question; headings only where there are no dates)

- BFS/DFS Graph and Tree Traversal and Shortest Paths (7)
- Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns (9)
- Dynamic Programming and State-Space Optimization (4)
- Expression Parsing and Evaluation (4)
- Other: Design, Heaps, Intervals and Misc (9)

Sources

- hacktherounds.com/questions/google
- prachub.com Google SWE Coding & Algorithms, Technical Screen only
- aced.io/questions?company=google&role=swe&type=coding
- 1point3acres.com/interview/problems/company/google (SWE coding, no OA)

How to use

- Tap any title in the Contents to jump to it.
- Every page has < Prev | Contents | Next > at the bottom.
- Prev and Next move one question at a time; Contents returns here.

Google - Contents

33 questions | tap a title to jump to it

1POINT3ACRES

22 questions | last 30 days: 0 | last 90 days: 22 Go >

ASKED: 31 TO 90 DAYS AGO (22) Go >

Medium (22) Go >

	M	1. Minimum-Cost Tree Disconnection	BFS / DFS / TREE		Sep 9 '26	TALK | 1Point3Acres
	M	2. Rotten Oranges / Multi-Source BFS (taxis)	BFS / DFS / TREE	IN APP	Sep 8 '26	TYPE | LC 994 | 1Point3Acres
	M	3. Longest Same-Color Grid Sequence	DP		Sep 8 '26	TALK | LC 562 | 1Point3Acres
	M	4. Compute File System Total Size	BFS / DFS / TREE		Sep 2 '26	READ | 1Point3Acres
	M	5. Next-Word Predictor	DESIGN / OTHER		Sep 2 '26	READ | 1Point3Acres
	M	6. Implement Python's itertools.tee	DESIGN / OTHER		Aug 20 '26	READ | 1Point3Acres
	M	7. Cutting Ribbons (LC 1891)	ARRAY / STRING		Aug 16 '26	TALK | LC 1891 | 1Point3Acres
	M	8. Union of Sorted Interval Lists	ARRAY / STRING		Aug 16 '26	TALK | LC 986 | 1Point3Acres
	M	9. Nested List Weight Sum (LC 339) with Follow-Ups	EXPRESSION		Aug 16 '26	TALK | LC 339 | 1Point3Acres
	M	10. Template Variable Expansion with Follow-Ups	BFS / DFS / TREE		Aug 14 '26	READ | 1Point3Acres
	M	11. Target Expressions from Ordered Digits	EXPRESSION		Aug 13 '26	TALK | LC 282 | 1Point3Acres
	M	12. Progressive Array Partitioning	ARRAY / STRING		Aug 11 '26	TALK | 1Point3Acres
	M	13. 2D Points Connected Components by Radius	BFS / DFS / TREE		Aug 4 '26	TALK | LC 547 | 1Point3Acres
	M	14. Streaming Median (LC 295) with Follow-ups	DESIGN / OTHER	IN APP	Aug 3 '26	TYPE | LC 295 | 1Point3Acres
★	M	15. Decode String	EXPRESSION	IN APP	Jul 29 '26	TYPE | LC 394 | 1Point3Acres
	M	16. Batch and Streaming Log Deduplication	DESIGN / OTHER		Jul 23 '26	READ | 1Point3Acres
	M	17. Mahjong Winning Hand Detection (14 Tiles)	ARRAY / STRING		Jul 20 '26	READ | 1Point3Acres
	M	18. Minimum Direction Violations in a Directed Graph	BFS / DFS / TREE		Jul 19 '26	TALK | LC 1466 | 1Point3Acres
★	M	19. Top-K Frequent in Large Logs	DESIGN / OTHER	IN APP	Jul 19 '26	TYPE | LC 347 | 1Point3Acres
	M	20. Count Word Decompositions into Chemical Ele...	DP		Jul 14 '26	TYPE | LC 139 | 1Point3Acres
	M	21. Piano Hand Movement Grouping	DESIGN / OTHER		Jul 14 '26	READ | 1Point3Acres
	M	22. Snake Case to Camel Case with Repository-Wi...	DESIGN / OTHER		Jul 14 '26	READ | 1Point3Acres

PRACHUB

7 questions | last 30 days: 0 | last 90 days: 7 Go >

POSTED: 31 TO 90 DAYS AGO (7) Go >

Hard (2) Go >

	H	23. Smallest Missing Positive Integer in Linear Tim...	ARRAY / STRING	IN APP	Sep 9 '26	TYPE | LC 41 | PracHub
	H	24. Find the Longest Straight Run of One Color	DP		Aug 23 '26	TALK | LC 562 | PracHub

Medium (4) Go >

	M	25. Sliding-Window Mean After Removing the Larg...	ARRAY / STRING		Sep 1 '26	TALK | LC 480 | PracHub
	M	26. Implement a Reusable Locker Allocation System	DESIGN / OTHER		Aug 14 '26	TALK | LC 1845 | PracHub
	M	27. Count Prefix-Matching Simple Paths in a Label...	BFS / DFS / TREE		Aug 9 '26	TYPE | LC 79 | PracHub
★	M	28. Expand Recursive Key-Value Placeholders in a...	EXPRESSION		Aug 4 '26	TYPE | LC 394 | PracHub

Easy (1) Go >

	E	29. Check an Interval Against Existing Intervals for...	ARRAY / STRING	IN APP	Aug 8 '26	TALK | LC 729 | PracHub

Google - Contents

33 questions | tap a title to jump to it

EXPONENT

4 questions | last 30 days: 0 | last 90 days: 4 Go >

ASKED (ROUGH): 31 TO 90 DAYS AGO (4) Go >

Hard (1) Go >

H 30. Merge k sorted linked lists. DESIGN / OTHER IN APP ~Aug '26 TYPE | LC 23 | Exponent

Medium (3) Go >

★ M 31. Merge Intervals ARRAY / STRING IN APP ~Aug '26 TYPE | LC 56 | Exponent

★ M 32. Group anagrams ARRAY / STRING IN APP ~Aug '26 TYPE | LC 49 | Exponent

★ M 33. Given n houses in a line with money in each, fin... DP IN APP ~Aug '26 TYPE | LC 198 | Exponent

SIDE NOTE: marked problems that are not in this book

★ The top 75 Medium and Hard problems and the top 10 Easy problems by Google frequency are marked in this book. These 39 are marked too but are not questions here. Do them from your app. Rank is the position in the Medium and Hard list, or E1 to E10 for Easy.

Rank	LC	Problem	Lvl	Freq
3	3	Longest Substring Without Repeating	M	73/69.3
5	42	Trapping Rain Water	H	73.6/55.1
7	128	Longest Consecutive Sequence	M	70.3/58.3
8	15	3Sum	M	72.6/58.3
9	7	Reverse Integer	M	66.3/51.4
15	11	Container With Most Water	M	68.3/41.1
18	410	Split Array Largest Sum	H	57.7/46.9
21	17	Letter Combinations of a Phone Number	M	57.8/51.4
25	239	Sliding Window Maximum	H	54.9/46.9
26	31	Next Permutation	M	61/41.1
30	6	Zigzag Conversion	M	53.5/51.4
31	34	Find First and Last Position	M	53/46.9
32	54	Spiral Matrix	M	59.3/33
34	1944	Number of Visible People in a Queue	H	53.6/41.1
37	33	Search in Rotated Sorted Array	M	57.2/33
38	84	Largest Rectangle in Histogram	H	56.8/33
39	167	Two Sum II	M	49.8/46.9
42	155	Min Stack	M	50.8/41.1
43	19	Remove Nth Node From End of List	M	50.2/41.1
44	424	Longest Repeating Character Replacement	M	53.6/33
47	739	Daily Temperatures	M	47.8/41.1
48	78	Subsets	M	51.7/33
49	39	Combination Sum	M	47.6/41.1
52	16	3Sum Closest	M	50.6/33
53	131	Palindrome Partitioning	M	46/41.1
54	209	Minimum Size Subarray Sum	M	45.2/41.1
57	402	Remove K Digits	M	44.1/0
62	151	Reverse Words in a String	M	46/33
63	312	Burst Balloons	H	41.8/0
67	51	N-Queens	H	58.6/0
75	210	Course Schedule II	M	36.5/33
E1	20	Valid Parentheses	E	67.3/70.9
E3	14	Longest Common Prefix	E	73.6/55.1
E4	88	Merge Sorted Array	E	69.7/55.1
E5	26	Remove Duplicates from Sorted Array	E	65.7/55.1
E7	28	Find the Index of the First Occurrence	E	56.2/61
E8	485	Max Consecutive Ones	E	56/58.3
E9	169	Majority Element	E	66.5/33
E10	242	Valid Anagram	E	57.3/51.4

Freq is LeetCode's frequency for Google: all time / last 30 days.

1POINT3ACRES

22 questions | Hard 0, Medium 22, Easy 0 | asked in the last 30 days: 0, last 90 days: 22 (grouped by recency, newest first) | topics: BFS/DFS and trees first

ASKED: 31 TO 90 DAYS AGO (22)

Medium (22)

MEDIUM | 1Point3Acres | phone-screen | 1 of 33

Minimum-Cost Tree Disconnection

ASKED 2026-09-09 | 31 DAYS AGO | LAST 90 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 193/224

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-09 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/binary-tree-root-leaf-minimum-cut

QUESTION

EXAMPLES

Negative-weight follow-up

```
A --(-1)--> B --(-1)--> C
Minimum total cut cost: -2
```

Cutting both edges costs -2, even though cutting just the first edge already separates the root from the original leaf.

APPROACH

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what must be disconnected (root from every leaf), whether the tree is rooted and directed, whether weights can be negative, and what to return for an empty or single-node tree.

BRUTE FORCE TO SAY OUT LOUD

Try every subset of edges and keep those that separate the root from all leaves. Exponential.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Define best(v) as the cheapest set of cuts in v's subtree that disconnects v from every leaf below it. For each child c with edge weight w, either cut the edge, paying w, or keep it and require best(c) (not allowed if c is a leaf). Sum over children. With non-negative weights that is the answer. With negative weights, once you cut an edge the whole subtree below becomes free, and you may still cut any negative edge there, so the cut option costs w plus the sum of min(0, edge) over that subtree.

DRY-RUN THESE CASES

A single edge, the negative chain A to B to C (answer -2), a leaf at different depths, a star.

LIKELY FOLLOW-UPS

Negative weights, many roots, or recursion depth for a chain tree.

PITFALLS

Treating every negative edge as a mandatory cut. Recursing deeply on a chain.

HOW TO THINK ABOUT IT

On a tree, every decision about a subtree depends only on the edge above it, so define a function of the node that returns the best cost inside its subtree. For each child you either cut the edge above it (pay its weight) or keep it and make the child itself disconnected from the leaves below. With negative weights, a cut edge frees the whole subtree, so you may cut every negative edge in it for free gain.

Basis: Written. Assumed setup (confirm it): a rooted tree with edge weights (cuts cost the weight, weights may be negative); cut edges so the root is disconnected from every leaf; a leaf is a non-root node with no children; with no leaves the cost is 0.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(root, edges):
    best = INF
    for k in range(len(edges) + 1):
        for cut in combinations(edges, k):
            tree = remove_edges(edges, cut)
            if not reaches_leaf(tree, root):
                cost = sum(w for _, _, w in cut)
                best = min(best, cost)
    return best
```

TIME	SPACE
O(2^E * n)	O(n)

WHY IT IS SLOW

2^E subsets, each checked with a search: O(2^E * n). A tree DP (cut the edge above a child, or require the child to be disconnected from its leaves) is O(n).

OPTIMAL SOLUTION · Tree DP: best(v) = cheapest cuts inside v's subtree that disconnect v from its leaves

TIME	SPACE
O(n)	O(h)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Weights are non-negative.

Then free(c) is always 0 and the recurrence collapses to best(v) = sum over children of (w if the child is a leaf, else min(w, best(child))).

FOLLOW-UP 2 The tree is a long chain and recursion overflows.

Compute nodes in reverse BFS order (children before parents) and store best and free in arrays, no recursion needed. State the space as O(n).

FOLLOW-UP 3 Return which edges to cut.

Remember at each child which option was cheaper, then walk down from the root collecting the cut edges. Cut edges also include every negative edge below any cut edge.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
from collections import defaultdict

INF = float("inf")


def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root
    kids = defaultdict(list)
    for u, v, w in edges:
        kids[u].append((v, w))
    # FOLLOW-UP 2: a loop, not recursion: safe on a long
    # chain
    # breadth-first order: parents come before children
    order = [0]
    for u in order:
        for v, _ in kids[u]:
            order.append(v)
    # best[v]: cheapest way to cut v off from its leaves
    # free[v]: sum of every negative cost below v
    best, free = [0] * n, [0] * n
    for v in reversed(order):
        for c, w in kids[v]:
            # 1) cut this edge and the negatives below
            cut = w + free[c]
            # 2) keep it and cut deeper (a leaf cannot)
            keep = best[c] if kids[c] else INF
            best[v] += min(cut, keep)
            free[v] += min(0, w) + free[c]
    return kids, best, free


# BASE QUESTION (optimal): Tree DP: best(v) = cheapest
# cuts inside v's subtree that ...
# FOLLOW-UP 1: Weights are non-negative.
# FOLLOW-UP 2: The tree is a long chain and recursion
# overflows.
def disconnect_cost(n, edges):
    # works for negative costs and needs no recursion
    return tree_costs(n, edges)[1][0]


# FOLLOW-UP 3: Return which edges to cut.
def edges_to_cut(n, edges):
    kids, best, free = tree_costs(n, edges)
    cuts, stack = [], [0]
    while stack:
        v = stack.pop()
        for c, w in kids[v]:
            keep = best[c] if kids[c] else INF
            if w + free[c] <= keep:
                # cut here, plus negative edges below c
                cuts.append((v, c))
                below = [c]
                while below:
                    x = below.pop()
                    for y, wy in kids[x]:
                        if wy < 0:
                            cuts.append((x, y))
                        below.append(y)
            else:
                stack.append(c)
    return cuts


if __name__ == "__main__":
    # the question's chain A -(-1)-> B -(-1)-> C
    neg = [(0, 1, -1), (1, 2, -1)]
    assert disconnect_cost(3, neg) == -2
    cuts = sorted(edges_to_cut(3, neg))
    assert cuts == [(0, 1), (1, 2)]
    # non-negative costs: cut the cheaper edge
    chain = [(0, 1, 5), (1, 2, 3)]
    assert disconnect_cost(3, chain) == 3
    assert edges_to_cut(3, chain) == [(1, 2)]
    # a single node needs no cut
    assert disconnect_cost(1, []) == 0
    # a long chain does not overflow the stack
    big = [(i, i + 1, 1) for i in range(5000)]
    assert disconnect_cost(5001, big) == 1
```

NOTES FROM CANDIDATES

Clarify the required result for an empty tree and a single-node tree. No final return convention is specified for these cases. Include recursion-stack memory in space analysis. The tree can degenerate into a chain; balanced-tree depth is not a worst-case bound.
Coding took place directly in Google Docs with no supplied framework; the candidate also had to define the tree node structure.
The interviewer asked the candidate to think aloud while developing the approach.
For nonnegative weights, use postorder DP. Let F(v) be the minimum cost to disconnect node v from every original leaf below it. An original leaf has F(v) = infinity; an internal node has F(v) = sum(min(weight(v,c), F(c))) over its children. Each branch must be blocked independently. Use a safe infinity representation rather than adding to a fixed-width maximum integer.
For negative weights, cutting an ancestor edge does not remove the opportunity to cut negative edges below it. Track A(v), the minimum optional cut cost when the subtree is already disconnected: A(v) = sum(min(0, weight(v,c)) + A(c)), with A(leaf) = 0. Then use F(v) = sum(min(weight(v,c) + A(c), F(c))), retaining F(leaf) = infinity. This includes beneficial redundant negative cuts

without double counting edges.
Both recurrences process each edge once: O(n) time and O(h) recursive auxiliary space, where h can equal n. A single original root-leaf cannot be disconnected from itself by cutting edges; agree on its result convention before coding.

PREPARATION

Implement the nonnegative postorder recurrence, then trace a chain and a branching tree while accounting for recursion depth. Implement the two-state negative-weight version and compare it with exhaustive edge-subset enumeration on small trees, including the given negative chain.

MEDIUM | IN APP LC 994 | 1Point3Acres | onsite-coding, phone-screen | 2 of 33

Rotten Oranges / Multi-Source BFS (taxis)

ASKED 2026-09-08 | 32 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 51.2** | **30 DAYS 0** | **PRIORITY 66/224**

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths
Optimal answer from: written and tested here for this exact question
This question has several parts, so each part has its own answer below.
Also in your app: LeetCode #994 Rotting Oranges
LeetCode match: LC 994 Rotting Oranges

ABOUT THIS QUESTION

Frequency: Low | Last asked: 2026-09-08 | Roles: SWE, MLE
Original: https://www.1point3acres.com/interview/problems/company/google/rotten-oranges-multi-source-bfs-taxis

QUESTION

EXAMPLES

```
Grid [[2,1,1],[1,1,0],[0,1,1]] (rotting-oranges
    variant): return number of minutes until all
    reachable 1s become 2 (4).
```

Taxi variant: for each empty cell, distance to nearest taxi.

APPROACH

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

HOW TO THINK ABOUT IT

'Spread outward from several starting points at the same time and count the rounds' is multi-source BFS: put every source in the queue before you start, and process the queue one round at a time. Rounds are minutes. Always count the fresh ones first so that unreachable oranges are detected at the end.

Basis: LeetCode 994, Rotting Oranges, and the same multi-source BFS skeleton as LeetCode 542 and 286.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    minutes = 0
    while True:
        fresh = fresh_next_to_rotten(grid)
        if not fresh:
            break
        for r, c in fresh:
            grid[r][c] = 2
        minutes += 1
    return -1 if any_fresh(grid) else minutes
```

TIME	SPACE
O(minutes * m * n)	O(m * n)

WHY IT IS SLOW

Each minute scans the entire grid, O(minutes * m * n). A queue of the newly rotten oranges touches each cell only once, O(m * n).

OPTIMAL SOLUTION · Multi-source BFS by minute

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Taxi variant: distance from every empty cell to the nearest taxi

TIME	SPACE
O(mn)	O(mn)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Sources move or new taxis appear. How do you keep the distances fresh?

Adding a source is cheap: push it at distance 0 and run a BFS that only continues into cells where it beats the stored distance (a decrease-only update, amortized small). Removing or moving a source is hard because distances can increase; either rebuild from scratch with a multi-source BFS, or rebuild periodically and accept bounded staleness in between.

FOLLOW-UP 2 Why is multi-source BFS the same cost as single-source BFS?

A virtual super-source connected to all sources by zero-length edges turns the problem into one BFS. Each cell enters the queue once, so it is O(mn) no matter how many sources exist.

FOLLOW-UP 3 Signal variant: a signal spreads from sources, how long until every target gets it?

Run the BFS and take the maximum distance over the target cells. If any target is unreachable the answer is infeasible (-1); with no targets it is 0. To allow destroying at most one blocker, compute distances from the sources and also from the targets and test each blocker cell as a bridge.

FOLLOW-UP 4 A signal spreads from sources to targets: how long until every target is reached?

One multi-source BFS from the sources, then the answer is the largest distance over the targets (-1 if any target is unreachable, 0 if there are no targets). Per-source BFS would cost O(mn * sources).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import deque


def multi_source_bfs(rows, cols, sources, open_cell):
    # distance from the nearest source to every cell
    dist = {s: 0 for s in sources}
    queue = deque(sources)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c),
                       (r, c + 1), (r, c - 1)):
            inside = 0 <= nr < rows and 0 <= nc < cols
            if inside and (nr, nc) not in dist:
                if open_cell(nr, nc):
                    dist[(nr, nc)] = dist[(r, c)] + 1
                    queue.append((nr, nc))
    return dist


# BASE QUESTION (optimal): Multi-source BFS by minute
def rotting_minutes(grid):
    rows, cols = len(grid), len(grid[0])
    cells = [(r, c) for r in range(rows)
             for c in range(cols)]
    rotten = [x for x in cells if grid[x[0]][x[1]] == 2]
    fresh = [x for x in cells if grid[x[0]][x[1]] == 1]
    # rot spreads through any non-empty cell
    dist = multi_source_bfs(
        rows, cols, rotten, lambda r, c: grid[r][c] != 0)
    # a fresh orange that is never reached means -1
    if any(x not in dist for x in fresh):
        return -1
    return max((dist[x] for x in fresh), default=0)


# TAXI VARIANT: distance to the nearest taxi
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked
    rows, cols = len(grid), len(grid[0])
    taxis = [(r, c) for r in range(rows)
             for c in range(cols) if grid[r][c] == "T"]
    dist = multi_source_bfs(
        rows, cols, taxis, lambda r, c: grid[r][c] != "#")
    # None where no taxi can reach the cell
    return [[dist.get((r, c)) for c in range(cols)]
            for r in range(rows)]


# FOLLOW-UP 4: A signal spreads from sources to targets:
# how long until every target ...
def signal_time(rows, cols, sources, targets, blocked=()):
    # how long until every target hears the signal?
    dist = multi_source_bfs(
        rows, cols, list(sources),
        lambda r, c: (r, c) not in blocked)
    if any(t not in dist for t in targets):
        return -1
    # no targets means no waiting
    return max((dist[t] for t in targets), default=0)


if __name__ == "__main__":
    assert rotting_minutes([[2, 1, 1], [1, 1, 0],
                            [0, 1, 1]]) == 4
    # a fresh orange walled off by an empty cell
    assert rotting_minutes([[2, 0, 1]]) == -1
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [
        [0, 1, None], [1, 2, 3]]
    # signal from two sources to two targets
    assert signal_time(1, 5, [(0, 0), (0, 4)],
                       [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)],
                       blocked={(0, 1)}) == -1
```

```
assert signal_time(1, 3, [(0, 0)], []) == 0
```

NOTES FROM CANDIDATES

BFS from all sources simultaneously is O(mn); per-source BFS is O(mn × #sources).
The moving-source follow-up trades exactness for amortized cost — common answer is periodic rebuild with bounded staleness.
For the signal variant, use the maximum source-distance over targets as the completion time; an unreachable target makes that candidate infeasible. Agree that no targets requires zero time. A correct baseline for destroying at most one blocker is to run multi-source BFS with no deletion, then once with each blocker made traversable, restoring it before the next trial. Minimize the completion time across feasible trials; return -1 if none is feasible. With B blockers on an R by C grid, this takes O((B + 1) * R * C) time and O(R * C) reusable auxiliary space. A BFS state that allows each path to break its own blocker does not enforce one shared destroyed blocker for all targets.

PREPARATION

Drill multi-source BFS from memory in under 10 min.
Prep a 2-minute pitch on incremental-vs-rebuild trade-off for the moving-source follow-up.
Practice the rotting-oranges / walls-and-gates / 01-matrix triplet — all share the multi-source BFS skeleton.

MEDIUM | 1Point3Acres | phone-screen | 3 of 33

Longest Same-Color Grid Sequence

ASKED 2026-09-08 | 32 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 147/224

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix (related)

Frequency: Single report | Last asked: 2026-09-08 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

QUESTION

Given a grid of colors, return the longest consecutive same-color sequence along a horizontal, vertical, or diagonal line.

Input is a two-dimensional grid of colors such as R, G, and B. Return the length of the longest consecutive sequence of a single color horizontally, vertically, or diagonally.

QUESTION

EXAMPLES

```
R R R G
R R B B
G R R R
B G B R

Answer: 4
```

APPROACH

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

HOW TO THINK ABOUT IT

A straight line can run in four directions (right, down, down-right, down-left). For each cell and each direction keep the length of the same-colour run that ends there: it is the predecessor's run plus one if the colours match, otherwise 1. Because every run only looks one cell back in a fixed direction, one pass in reading order is enough.

Basis: LeetCode 562, Longest Line of Consecutive One in Matrix, generalised to any colour.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(grid):
    best = 0
    for r, c in all_cells(grid):
        for dr, dc in DIRECTIONS:
            n = 1
            while same(grid, r, c,
                       r + n * dr, c + n * dc):
                n += 1
            best = max(best, n)
    return best
```

TIME	SPACE
O(m * n * max(m, n))	O(1)

WHY IT IS SLOW

Re-walks the same cells in every line, O(m * n * max(m, n)). Keeping the run length ending at each cell in each direction is O(m * n).

OPTIMAL SOLUTION · Four run lengths per cell

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The line may also bend (a connected region).

Then it is a different problem: the largest connected same-colour region, found with BFS/DFS or union-find in O(m * n). The run-length DP only handles straight lines because each run depends on exactly one predecessor.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Four run lengths per cell
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n, best = len(g[0]), 0
    # run lengths ending in the previous row:
    # straight down, down-left, down-right
    up, dl, dr = [0] * n, [0] * n, [0] * n
    for i, row in enumerate(g):
        nu, nl, nr = [1] * n, [1] * n, [1] * n
        left = 0
        for j, color in enumerate(row):
            # horizontal run from the cell on the left
            same = j and row[j - 1] == color
            left = left + 1 if same else 1
            if i:
                # extend the run from the cell above
                if g[i - 1][j] == color:
                    nu[j] = up[j] + 1
                # from above-left (down-right diagonal)
                if j and g[i - 1][j - 1] == color:
                    nr[j] = dr[j - 1] + 1
                # from above-right (down-left diagonal)
                if j + 1 < n and g[i - 1][j + 1] == color:
                    nl[j] = dl[j + 1] + 1
            best = max(best, left, nu[j], nl[j], nr[j])
        up, dl, dr = nu, nl, nr
    return best


# FOLLOW-UP 1: The line may also bend (a connected
# region).
def largest_region(g):
    # follow-up: the line may bend (connected region)
    seen, best = set(), 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack, size = [(r, c)], 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y),
                             (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0:
                        continue
                    if a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen \
                            and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            best = max(best, size)
    return best


if __name__ == "__main__":
    grid = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_same_color_line(grid) == 4
    assert longest_same_color_line([]) == 0
    # one column and alternating colours
    assert longest_same_color_line(["R", "R", "R"]) == 3
    assert longest_same_color_line(["RGR", "GRG"]) == 2
    # a bent region is longer than any straight line
    bent = ["RRR", "GGR"]
    assert longest_same_color_line(bent) == 3
    assert largest_region(["RR", "GR"]) == 3
```

NOTES FROM CANDIDATES

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.
Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.
Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

PREPARATION

Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

MEDIUM | 1Point3Acres | onsite-coding, phone-screen | 4 of 33

Compute File System Total Size

ASKED 2026-09-02 | 38 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 200/224

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/file-system-total-size

QUESTION

EXAMPLES

```
root = Dir(
  Dir(File(10), File(20)),
  File(5),
  Dir(File(30)),
)
Total: 65
```

APPROACH

Drill recursive-and-iterative DFS over directory-style trees. Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the structure can contain cycles or shared directories (links), whether files have sizes only, and whether to handle very deep trees.

BRUTE FORCE TO SAY OUT LOUD

Recursive DFS adding file sizes. Simple and usually the expected first answer.

OPTIMAL IDEA

DFS with a visited set if cycles or shared nodes are possible, so each node is counted once. Use an explicit stack for deep trees. If queried repeatedly, memoize directory totals and invalidate ancestors when a file changes. O(n) per computation.

DRY-RUN THESE CASES

The example giving 65, an empty directory, a directory containing only directories, a symlink loop.

LIKELY FOLLOW-UPS

Cache totals, handle updates, or find the largest directory.

PITFALLS

Recursion depth on a long chain. Double counting shared nodes.

HOW TO THINK ABOUT IT

A total over a tree is a post-order sum. Two things change the answer in real file systems: depth (recursion limit) and sharing or cycles (the same node reachable twice must be counted once). A visited set solves both the cycle risk and the double counting, and an explicit stack solves the depth.

Basis: Written; a tree sum (like LeetCode 690 / 1448 style traversals). Assumed model: File(size) and Dir(children...); directories may be shared (links) in the harder version.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def size(node):
    if is_file(node):
        return node.size
    total = 0
    for child in node.children:
        total += size(child)
    return total
```

TIME	SPACE
O(n)	O(depth) call stack

WHY IT IS SLOW

Linear time, which is optimal; the weaknesses are the recursion depth on very deep trees and an endless loop if a link creates a cycle. An explicit stack with a visited set fixes both.

OPTIMAL SOLUTION · Iterative traversal with a visited set

TIME	SPACE
O(n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The tree is queried often and files change.

Cache each directory's total and, when a file changes, clear the cache only for its ancestors (walk parent links). A query recomputes only cleared nodes. Updates cost O(depth).

FOLLOW-UP 2 Hard links: the same file appears under two directories.

Count a file once per identity (a visited set of ids, as coded) if the question asks for disk usage, but once per path if it asks for the sum of the sizes shown. Ask which.

FOLLOW-UP 3 Find the largest directory.

Compute totals in post-order and keep the maximum; it is the same pass. For top-K directories keep a heap of size K. **Cost:** one post-order pass, O(n); keeping the top K directories with a heap of size K makes it O(n log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq


class File:
    def __init__(self, size):
        self.size, self.parent = size, None
```

```
class Dir:
    def __init__(self, *children):
        self.children, self.parent = list(children), None
        # cached total; None means "needs recomputing"
        self.cached = None
        for child in children:
            child.parent = self


# BASE QUESTION (optimal): Iterative traversal with a
# visited set
# FOLLOW-UP 2: Hard links: the same file appears under two
# directories.
def total_size(root, count_once=True):
    # iterative walk; count_once handles hard links
    total, seen, stack = 0, set(), [root]
    while stack:
        node = stack.pop()
        # FOLLOW-UP 2: hard links: count a shared file
        # once
        if count_once:
            if id(node) in seen:
                continue
            seen.add(id(node))
        if isinstance(node, File):
            total += node.size
        else:
            stack.extend(node.children)
    return total


def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached
```

```
def cached_total(root):
    # post-order: a directory after all its children
    order, stack = [], [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            stack.extend(node.children)
    for node in reversed(order):
        if isinstance(node, Dir) and node.cached is None:
            kids = node.children
            node.cached = sum(size_of(c) for c in kids)
    return root.cached


# FOLLOW-UP 1: The tree is queried often and files change.
def resize(file, size):
    # a file changed: only its ancestors go stale
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent


# FOLLOW-UP 3: Find the largest directory.
def directory_totals(root):
    # the size of every directory, in one pass
    cached_total(root)
    out, stack = [], [root]
    while stack:
        node = stack.pop()
        if isinstance(node, Dir):
            out.append(node.cached)
            stack.extend(node.children)
    return out


# FOLLOW-UP 3: Find the largest directory.
def top_k_directories(root, k):
    # keep a heap of size k: O(n log k)
    return heapq.nlargest(k, directory_totals(root))
```

```
if __name__ == "__main__":
    inner = Dir(File(10), File(20))
    root = Dir(inner, File(5), Dir(File(30)))
    assert total_size(root) == 65
    assert cached_total(root) == 65
    # change one file: only its ancestors are recomputed
    first = inner.children[0]
    resize(first, 15)
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70
    assert top_k_directories(root, 2) == [70, 35]
    # a hard link: the same file under two directories
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14
```

NOTES FROM CANDIDATES

If the interviewer says "strict tree", drop the visited set and stick with plain DFS — extra cycle handling reads as over-engineering.
For the cache follow-up, store the cached total on each directory node; when a file changes, walk parent pointers invalidating along the path.
The interviewer also pushes for an iterative DFS (explicit stack) after the recursive solution lands — have both ready.

PREPARATION

Drill recursive-and-iterative DFS over directory-style trees.
Have ready: cycle-safe DFS (visited set), memoized DFS (cache on internal nodes), invalidation cascade.
Practice the same problem under the the canonical "longest absolute file path" parsing problem for warm-up.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 5 of 33

Next-Word Predictor

ASKED 2026-09-02 | 38 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 220/224

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-09-02 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/next-word-frequency-predictor

QUESTION

Process training data and predict the most frequent next word for a supplied word. Maintain prediction state during processing so prediction can run in constant time.SWEstringhashmapdata-structureFrequencySingle reportLast asked2026-09-02Stageonsite-coding

APPROACH

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask how sentences are split and cased, how ties are broken, what to return for an unseen word, and whether training is incremental.

BRUTE FORCE TO SAY OUT LOUD

Store all sentences and scan them for each prediction. O(corpus) per query.

OPTIMAL IDEA

Keep a dict from a word to a dict of next-word counts, and a separate best[word] updated whenever a count changes (if the new count beats the current best, or ties and wins the tie-break). Prediction is a dict lookup, O(1). Training is O(total words).

DRY-RUN THESE CASES

A repeated pair, a tie, a new leader taking over, a word with no successor, the last word of a sentence.

LIKELY FOLLOW-UPS

Top-k predictions (keep a heap per word), bigram versus trigram context, or decay for recent data.

PITFALLS

Updating best only on strict increase and ignoring the tie rule. Counting across sentence boundaries.

HOW TO THINK ABOUT IT

Prediction in O(1) means the answer must already be sitting in a table when the query arrives, so do the work while training. Keep counts per (word, next word) and, for each word, the current best successor. Each count only increases by one, so the best can be updated by comparing just the one count that changed.

Basis: Written; a bigram language model. Assumed rules (confirm them): words are split on spaces and compared case-sensitively; pairs never cross sentence boundaries; ties go to the alphabetically smaller word; an unseen word predicts None.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
sentences = []
def train(sentence):
    sentences.append(sentence.split())
def predict(word):
    counts = Counter()
    for words in sentences:
        for a, b in zip(words, words[1:]):
            if a == word:
                counts[b] += 1
    if not counts:
        return None
    return max(counts, key=counts.get)
```

TIME	SPACE
O(corpus size) per prediction	O(corpus size)

WHY IT IS SLOW

Each prediction is a full pass over the corpus. Counting pairs while training and keeping the best successor per word makes prediction O(1).

OPTIMAL SOLUTION · Counts plus a maintained best successor

TIME	SPACE
O(1) per pair when training and per prediction	O(number of distinct pairs)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return the top K predictions.

Keep the Counter per word and use heapq.nsmallest on (-count, word): O(d log K) per query, or keep a sorted structure per word if queries dominate.

FOLLOW-UP 2 Use two words of context (trigrams).

The key becomes the pair (previous two words) and everything else is identical, at the price of more memory. Back off to the bigram table when the pair was never seen.

FOLLOW-UP 3 The corpus keeps changing and old text should fade.

Use an exponentially decayed count (multiply by a factor per training step, or store a timestamp and decay on read), and recheck the best when a count drops below another.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import Counter, defaultdict


# BASE QUESTION (optimal): Counts plus a maintained best
# successor
# FOLLOW-UP 1, 2: Return the top K predictions.
class Predictor:
    # FOLLOW-UP 2: orders=(2, 1) means two words of
    # context
    def __init__(self, orders=(2, 1)):
        # orders: how many previous words are the context
        self.orders = sorted(orders, reverse=True)
        # order -> context tuple -> Counter of next words
        self.counts = {k: defaultdict(Counter)
                       for k in orders}
        # order -> context tuple -> best next word so far
        self.best = {k: {} for k in orders}

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                ctx = tuple(words[i:i + k])
                nxt = words[i + k]
                self.update(k, ctx, nxt)

    def update(self, k, ctx, nxt):
        table = self.counts[k][ctx]
        table[nxt] += 1
        cur = self.best[k].get(ctx)
        # more counts win; a tie goes to the smaller word
        better = cur is None or table[nxt] > table[cur]
        tie = cur is not None and table[nxt] == table[cur]
        if better or (tie and nxt < cur):
            self.best[k][ctx] = nxt

    def predict(self, *context):
        # O(1): try the longest context, then back off
        for k in self.orders:
            if len(context) >= k:
                ctx = tuple(context[len(context) - k:])
                if ctx in self.best[k]:
                    return self.best[k][ctx]
        return None
```

```
    # FOLLOW-UP 1: the top K predictions
    def top_k(self, word, k):
        # k most likely next words after one word
        by_ctx = self.counts.get(1, {})
        table = by_ctx.get((word,), Counter())
        return heapq.nsmallest(
            k, table, key=lambda w: (-table[w], w))


if __name__ == "__main__":
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    # "love" is followed by cats twice, dogs once
    assert p.predict("love") == "cats"
    # two words of context are tried first
    assert p.predict("i", "love") == "cats"
    # an unseen pair backs off to the last word
    assert p.predict("we", "love") == "cats"
    assert p.predict("zebra") is None
    assert p.top_k("love", 2) == ["cats", "dogs"]
    # a tie goes to the alphabetically smaller word
    q = Predictor()
    q.train("a z")
    q.train("a b")
    assert q.predict("a") == "b"
```

NOTES FROM CANDIDATES

Exact method signatures, tokenization rules, frequency ties, and behavior for an unseen word are unspecified.
The candidate used a dictionary and stored the current best successor during processing. Calling this state a cache caused a misunderstanding: the interviewer initially thought updates would happen during prediction. Explain the update timing explicitly.
After agreeing on tokenization and sequence boundaries, increment a counter for each adjacent pair (previous_word, next_word). For each previous word, retain a successor whose count is maximal. Compare the updated successor with the retained best immediately during processing; prediction only reads that stored best.
With insertion-only counts, a pair whose count did not change cannot newly overtake the best. Under a practice convention that retains the incumbent on ties, replace the best only when the updated count is strictly larger. If a different tie rule is required, apply it explicitly on equal counts.
Hash-table updates per observed pair and prediction are expected O(1), excluding tokenization/string hashing costs. Store O(P + V) state for P distinct pairs and V predecessor words. Clarify unseen-word results and whether separate training sequences create cross-boundary pairs.

PREPARATION

Implement pair counting and best-successor maintenance during processing, then compare predictions with a full scan of each successor counter.
Trace repeated pairs, a tie, a new leader, an unseen word, and separate training sequences; write down the tie and sequence-boundary conventions before testing.

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 6 of 33

Implement Python's itertools.tee

ASKED 2026-08-20 | 51 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 221/224

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-20 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/python-tee-independent-iterators

QUESTION

Implement the behavior of Python's **`tee`**: accept one iterator and an integer **n**, then return **n** independent iterators over the same source sequence. Define the required function and classes yourself and improve a naive design toward an efficient shared-state implementation.SWEiteratorqueuedata-structureobject-designmediumFrequencySingle reportLast asked2026-08-20Stageonsite-coding

APPROACH

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the source iterator can be consumed by one consumer first, what happens if one consumer is far ahead, and whether thread safety is needed.

BRUTE FORCE TO SAY OUT LOUD

Store the whole source in a list and give each consumer an index. Simple, but defeats the purpose for large or infinite iterators.

OPTIMAL IDEA

Keep a deque per consumer. When a consumer needs an item and its deque is empty, pull the next item from the source and append it to every consumer's deque. Each item is held only until all consumers have read it. Build with a shared state object and a small iterator class with __iter__ and __next__. The linked-list variant uses one chain of nodes with each consumer holding a pointer, which frees nodes as the slowest consumer moves.

DRY-RUN THESE CASES

Two consumers at different speeds, n = 1, n = 0, an empty source, an exhausted source.

LIKELY FOLLOW-UPS

INTERVIEW-DAY PLAN

Thread safety (a lock around the pull), and memory growth if one consumer stalls.

PITFALLS

Pulling from the source once per consumer instead of once per item. Not appending to the other deques.

HOW TO THINK ABOUT IT

The source can only be consumed once, so someone must remember items until every consumer has read them. The state to keep is exactly 'items the slowest consumer has not read yet', so either give each consumer a queue that the leader fills, or keep one shared linked list with a pointer per consumer so memory is freed automatically.

Basis: The itertools.tee documentation's recipe; the linked-list variant is how CPython shares buffers.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def tee(source, n):
    items = list(source)  # read it all now
    def reader():
        for x in items:
            yield x
    return [reader() for _ in range(n)]
```

TIME	SPACE
O(n) up front	**O(n)**

WHY IT IS SLOW

It reads the source completely before anything is consumed, never works on an infinite or lazy source, and keeps every item in memory. One buffer per consumer (or a shared linked list) holds only what the slowest consumer has not read.

OPTIMAL SOLUTION · One deque per consumer, filled lazily

TIME	SPACE
O(n) total, O(1) amortized per item per consumer	**O(lag between fastest and slowest consumer)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Memory grows when one consumer stalls. Is there a way to free items automatically?

Use one shared singly linked list with a pointer per consumer. The only references to old nodes are the consumers' pointers, so Python's garbage collector frees nodes behind the slowest consumer without any bookkeeping. This is the design CPython uses.

FOLLOW-UP 2 Make it safe for several threads.

Pull from the source under one lock and append to the queues inside it. Without the lock two consumers can pull two items and append them in different orders to different queues. **Cost:** one lock acquisition per item pulled, O(1) extra per item; contention grows with the number of consumers.

FOLLOW-UP 3 Why not just copy the iterator?

Most iterators (files, generators, sockets) cannot be copied. tee gives independent views by buffering, which is the only thing you can do when the source cannot rewind.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import threading
from collections import deque


# FIRST IDEA (simplest): copy into a list
def tee_naive(iterable, n=2):
    # simplest idea: copy everything into a list
    items = list(iterable)
    # but it never works for endless iterators
    return tuple(iter(items) for _ in range(n))


# BASE QUESTION (optimal): One deque per consumer, filled
# lazily
def tee_queues(iterable, n=2):
    source = iter(iterable)
    # one waiting line of values per consumer
    queues = [deque() for _ in range(n)]

    def consumer(mine):
        while True:
            if not mine:
                try:
                    value = next(source)
                except StopIteration:
                    return
                # give the new value to everybody
                for q in queues:
                    q.append(value)
            yield mine.popleft()

    return tuple(consumer(q) for q in queues)


# FOLLOW-UP 1: Memory grows when one consumer stalls. Is
# there a way to free items ...
class Cell:
    # one link of the shared list
    def __init__(self):
        self.filled = False
        self.value = None
        self.next = None


# FOLLOW-UP 2: Make it safe for several threads.
```

```
def tee(iterable, n=2):
    source = iter(iterable)
    # one lock keeps the pulls in a single order
    lock = threading.Lock()

    def consumer(cell):
        while True:
            with lock:
                if not cell.filled:
                    # the first reader pulls the value
                    try:
                        cell.value = next(source)
                    except StopIteration:
                        return
                    cell.filled = True
                    cell.next = Cell()
                value, cell = cell.value, cell.next
            yield value

    # every reader starts at the same first cell
    head = Cell()
    return tuple(consumer(head) for _ in range(n))


if __name__ == "__main__":
    a, b = tee(range(5))
    # readers are independent, at any speed
    assert next(a) == 0 and next(a) == 1
    assert list(b) == [0, 1, 2, 3, 4]
    assert list(a) == [2, 3, 4]
    x, y, z = tee_queues("abc", 3)
    assert list(x) == list(y) == list(z) == list("abc")
    # works on an endless source, which tee_naive cannot
    def count():
        i = 0
        while True:
            yield i
            i += 1
    e1, e2 = tee(count())
    assert [next(e1) for _ in range(3)] == [0, 1, 2]
    assert next(e2) == 0
    assert [list(i) for i in tee_naive([1, 2], 2)] == [
        [1, 2], [1, 2]]
```

NOTES FROM CANDIDATES

The round depends on Python iterator-protocol fluency as much as general data-structure reasoning. Clarify laziness, source exhaustion, invalid n, and whether concurrent consumption is in scope before implementing.

A correct baseline consumes the source lazily once and gives each returned iterator its own queue: when one queue is empty, pull one source value, append it to every queue, then return it from the requesting queue. If n iterators trail across a gap of g values, however, those queues can retain O(n·g) buffered copies. For the optimal shared-state design, keep one linked buffer (or indexed deque) of source values plus one cursor per iterator, and discard a buffered value only after every cursor has passed it. That preserves independent consumption with O(g+n) auxiliary state, where g is the distance between the fastest and slowest consumers.

PREPARATION

Implement a custom Python iterator class from memory, including __iter__ and __next__.
Trace two and three consumers advancing at different speeds and identify what state must remain available.
Compare a fully materialized baseline with a lazy shared-state design and explain the memory behavior.

MEDIUM | 1Point3Acres | onsite-coding | 7 of 33

Cutting Ribbons (LC 1891)

ASKED 2026-08-16 | 55 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 19.6 | 30 DAYS 0 | PRIORITY 146/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1891 Cutting Ribbons

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/cutting-ribbons-max-equal-length

QUESTION

LC 1891 Cutting Ribbons: given wood lengths and K, maximize the equal piece length that yields at least K pieces. Follow-ups cover the search bound, floating-point lengths, and why complexity depends on the value range rather than the number of elements.SWEArraybinary-searchcomplexity-analysismedium FrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether piece lengths must be integers, what to return if even length 1 fails (0), and the sizes of the array and k.

BRUTE FORCE TO SAY OUT LOUD

Try every length from the maximum down to 1 and count pieces. O(n * max).

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Feasibility is monotone: if you can cut k pieces of length L you can for any smaller length. Binary search L over [1, max]: count = sum(r // L), move right if count >= k. O(n log max). A tighter upper bound is min(max, sum // k).

DRY-RUN THESE CASES

k larger than the total length (0), one ribbon, all equal ribbons, k = 1 (answer is the max ribbon).

LIKELY FOLLOW-UPS

Real-valued lengths (fixed number of halvings, not an epsilon loop), why complexity depends on the value range, or an unlimited number of cuts.

PITFALLS

Overflow when summing. Searching from 0 and dividing by zero.

HOW TO THINK ABOUT IT

'Largest length such that at least K pieces fit' is a maximisation with a yes/no check. Larger lengths yield fewer pieces, so the check is monotone and you can binary search the answer. The check itself is one pass: sum of ribbon // length.

Basis: LeetCode 1891, Cutting Ribbons.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(ribbons, k):
    for L in range(max(ribbons), 0, -1):
        pieces = sum(r // L for r in ribbons)
        if pieces >= k:
            return L
    return 0
```

TIME	SPACE
O(n * max)	O(1)

WHY IT IS SLOW

Tries up to max(ribbon) lengths, each costing O(n); with ribbons up to 10^9 that is a billion probes. The check is monotone, so binary search needs only log2(max) probes.

OPTIMAL SOLUTION · Binary search the answer (integer lengths)

TIME	SPACE
O(n log(max length))	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why does the running time depend on the value range and not just on the number of ribbons?

Each probe costs O(n), and the number of probes is log2(max length) for the integer version. A huge length like 10^9 needs about 30 probes while 10 needs 4, even with the same n. For real numbers the probe count is set by the precision you promise, e.g. log2(max / epsilon).

FOLLOW-UP 2 What if the candidate length is given to you?

Then it is just the feasibility check: sum(r // L for r in ribbons) >= k, in O(n). The binary search exists only to find the best L; the check is the same function you reuse inside it.

FOLLOW-UP 3 What is a good upper bound for the search?

min(max(ribbons), sum(ribbons) // k). If the total length divided by k is smaller than the longest ribbon, no piece longer than that can ever give k pieces, which shrinks the range a little and avoids wasted probes. **Cost:** each probe is O(n); the tighter bound removes only a few of the log2(max) probes, so the total stays O(n log max).

FOLLOW-UP 4 Give the search a tighter upper bound.

No piece can be longer than the longest ribbon, and K pieces of length L need K * L total length, so L <= sum // K. Starting hi at min(max(ribbons), sum // K) saves a few probes and also returns 0 immediately when sum < K.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
def pieces(ribbons, length):
    # how many pieces of this length can we cut?
    return sum(r // length for r in ribbons)


# FOLLOW-UP 4: Give the search a tighter upper bound.
def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and
    # k pieces of length L need k * L total length
    return min(max(ribbons), sum(ribbons) // k)


# BASE QUESTION (optimal): Binary search the answer
# (integer lengths)
def max_length(ribbons, k):
    # the best whole-number length (0 if impossible)
    lo, hi, best = 1, upper_bound(ribbons, k), 0
    while lo <= hi:
        mid = (lo + hi) // 2
        if pieces(ribbons, mid) >= k:
            # mid works: remember it and try longer
            best, lo = mid, mid + 1
        else:
            hi = mid - 1
    return best
```

```python
# VARIANT: fractional lengths
def max_length_float(ribbons, k, rounds=50):
    # lengths may be fractions: fixed rounds of halving
    lo, hi = 0.0, float(max(ribbons))
    for _ in range(rounds):
        mid = (lo + hi) / 2
        # enough pieces: the answer is at least mid
        if sum(int(r // mid) for r in ribbons) >= k:
            lo = mid
        else:
            hi = mid
    return lo


if __name__ == "__main__":
    assert max_length([9, 7, 5], 3) == 5
    assert max_length([7, 5, 9], 4) == 4
    # not enough total length: no pieces at all
```

```python
    assert max_length([1, 2], 10) == 0
    # the check alone, when the length is given
    assert pieces([9, 7, 5], 5) == 3
    assert upper_bound([9, 7, 5], 3) == 7
    # fractional lengths: 3 pieces from one ribbon of 1
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 0 + 2) > 0.49
```

NOTES FROM CANDIDATES

The interviewer allowed the candidate to abandon an initial greedy/sorting direction, then unlocked the problem with one question: what changes if a candidate length is supplied? Expect the follow-ups, not the base implementation, to consume most of the round.
For the integer version, initialize the best answer to 0, search only positive candidate lengths, and return 0 when even length 1 cannot produce K pieces. The feasibility predicate is monotone: once a length is infeasible, every larger length is also infeasible.

PREPARATION

Implement the integer version from memory and prove the monotonic feasibility predicate.
Rework it with floating-point lengths and explain termination under both fixed-iteration and epsilon-based policies.
Practice stating the bound and complexity in terms of the numeric value domain without conflating it with input size.

MEDIUM | 1Point3Acres | onsite-coding | 8 of 33

Union of Sorted Interval Lists

ASKED 2026-08-16 | 55 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 25.5 | 30 DAYS 0 | PRIORITY 138/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #986 Interval List Intersections

LeetCode match: LC 986 Interval List Intersections (related)

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/sorted-interval-list-union

QUESTION

Reverse of LC 986: given two individually sorted, internally non-overlapping interval lists, return their union. Follow-ups extend to K lists with a heap, inputs too large for memory, and lists that may overlap internally.SWEintervaltwo-pointerheapstreamingmediumFrequencySingle reportLast asked2026-08-16Stageonsite-coding

APPROACH

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether intervals are closed or half-open, whether touching intervals merge, whether each list is sorted and non-overlapping internally (later follow-up removes this), and how large the inputs are.

BRUTE FORCE TO SAY OUT LOUD

Concatenate both lists, sort, and merge. O((m + n) log (m + n)).

OPTIMAL IDEA

Two-pointer merge: repeatedly take the interval with the smaller start from either list and fold it into the last output interval if it overlaps (start at most the previous end), else append it. O(m + n). For K lists use heapq.merge (a heap of K heads), and since it is a generator the inputs never need to fit in memory.

DRY-RUN THESE CASES

One list empty, no overlaps, one interval swallowing many, touching endpoints, identical lists.

LIKELY FOLLOW-UPS

K lists, lists too big for memory, lists that overlap internally (sort each first or heap everything).

INTERVIEW-DAY PLAN

PITFALLS

Using end instead of start to pick the next interval. Not taking the maximum of ends when merging.

HOW TO THINK ABOUT IT

Each list is already sorted, so you can walk both with two pointers like the merge step of merge sort: always take the interval with the smaller start next. Once the intervals come out in start order, merging is the usual one-pass fold: extend the last output interval if the next one overlaps or touches it, otherwise start a new one.

Basis: The reverse of LeetCode 986 (Interval List Intersections): union instead of intersection.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(list1, list2):
    merged = []
    for s, e in sorted(list1 + list2):
        if merged and s <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], e)
        else:
            merged.append([s, e])
    return merged
```

TIME	SPACE
O((m + n) log (m + n))	O(m + n)

WHY IT IS SLOW

Sorting ignores that both lists are already sorted: O((m + n) log (m + n)) instead of O(m + n) for a two-pointer merge.

OPTIMAL SOLUTION · Two-pointer merge

TIME	SPACE
O(m + n)	O(m + n) for the output

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 What if the lists are too big for memory?

Read each list through an iterator (a file or a paged API). union_k only ever holds one interval per list plus the current merged interval, so memory is O(K) and it can emit results as it goes. Sorted order is what makes streaming possible.

FOLLOW-UP 2 What if one list may overlap itself?

The two-pointer merge already works, because it folds every incoming interval into the last output interval. What breaks is only the assumption that each list is sorted by start; if that also fails, sort each list first (O(n log n)) or push everything through a heap.

FOLLOW-UP 3 Do touching intervals merge?

Decide the endpoint rule before coding. For closed intervals [1, 3] and [3, 5] share a point, so they merge (start <= last end). For half-open intervals [1, 3) and [3, 5) they do not overlap but are adjacent; whether to merge adjacent ones is a product choice, so ask.

FOLLOW-UP 4 Each list may overlap itself or be unsorted.

Normalise each list first (sort and merge it by itself, O(n log n)), after which the two-pointer union applies. Or sort everything together if memory allows.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq


# FOLLOW-UP 1: What if the lists are too big for memory?
def union_stream(lists, touching=True):
    # lists: sorted iterables of (start, end)
    merged = heapq.merge(*lists)
    cur = None
    for start, end in merged:
        if cur is None:
            cur = [start, end]
            continue
        # touching: [1, 3] and [3, 5] share the point 3
        joined = start < cur[1]
        if touching:
            joined = start <= cur[1]
        if joined:
            cur[1] = max(cur[1], end)
        else:
            # finished one block: emit it as we go
            yield tuple(cur)
            cur = [start, end]
    if cur is not None:
        yield tuple(cur)


# BASE QUESTION (optimal): Two-pointer merge
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# VARIANT: K lists with a heap
def union_k(lists, touching=True):
    # same code, any number of lists, O(K) memory
    return list(union_stream(lists, touching))


# FOLLOW-UP 2: What if one list may overlap itself?
def normalise(intervals):
    # a list that is unsorted or overlaps itself
    ordered = sorted(intervals)
    return list(union_stream([ordered]))


# FOLLOW-UP 4: Each list may overlap itself or be
```

```
# unsorted.
def union_unsorted(a, b):
    # fix each list first, then use the same union
    return union_two(normalise(a), normalise(b))


if __name__ == "__main__":
    a = [(1, 3), (6, 8)]
    b = [(2, 4), (9, 10)]
    assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
    # closed intervals that touch merge
    assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]
    # half-open intervals that only touch stay apart
    assert union_two([(1, 3)], [(3, 5)], False) == [
        (1, 3), (3, 5)]
    # three lists, one of them empty
    three = union_k([a, b, []])
    assert three == [(1, 4), (6, 8), (9, 10)]
    # a list that overlaps itself, and an unsorted one
    merged = union_unsorted([(5, 9), (1, 6)], [(8, 12)])
    assert merged == [(1, 12)]
    assert union_unsorted([(9, 10), (1, 2)], []) == [
        (1, 2), (9, 10)]
```

NOTES FROM CANDIDATES

The round intentionally layers one constraint at a time. Keep the base invariant explicit—every interval before the current pointer has already been absorbed into a canonical output prefix—so the streaming and K-way versions remain easy to justify.
Agree on endpoint semantics before coding: touching closed intervals merge, while touching half-open intervals do not overlap. For K streaming inputs, the heap plus one current interval per input uses O(K) working memory, excluding the output.

PREPARATION

Code the two-list merge without special-casing more than an empty result and a non-overlapping next interval.
Generalize the implementation to K iterators using a heap, then explain the memory bound.
Rehearse how the invariant changes when individual inputs are not internally disjoint.

MEDIUM | 1Point3Acres | onsite-coding | 9 of 33

Nested List Weight Sum (LC 339) with Follow-Ups

ASKED 2026-08-16 | 55 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 16.5 30 DAYS 0 PRIORITY 155/224

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 339 Nested List Weight Sum

Frequency: Single report | Last asked: 2026-08-16 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/nested-list-weighted-sum-parsing

QUESTION

EXAMPLES

```
"[3, 8, [2, 14], [2, [91]]]" -> 3×1 + 8×1 + 2×2 + 14×2
    + 2×2 + 91×3 = 320.
Use [2, [91]] as a manual trace for bracket depth and
    the final-number flush.
```

APPROACH

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the input is already parsed or a string, whether negatives and multi-digit numbers appear, and whether weight is depth from the top (1 at the outside) or the inverse weight variant.

BRUTE FORCE TO SAY OUT LOUD

Parse into a tree of lists and recurse with depth. Simple but recursion can overflow.

OPTIMAL IDEA

For a string, scan once with a depth counter: '[' adds one, ']' subtracts one, and an integer is multiplied by the current depth when its last digit ends (flush the number on any non-digit and at the end of the string). For parsed lists use BFS level by level. O(n), O(1) extra for the scanner.

DRY-RUN THESE CASES

[3, 8, [2, 14], [2, [91]]] gives 320. A number glued to the closing bracket. A negative number. A deeply nested list.

LIKELY FOLLOW-UPS

The inverse weight version (LeetCode 364), depth limit, or iterative versus recursive.

PITFALLS

Not flushing a trailing number. Treating '-' as a separate token.

HOW TO THINK ABOUT IT

The weight of a number is just the nesting depth where it appears, and the nesting depth is the number of '[' still open. That means you never need to build the nested structure: scan once with a counter, and flush a number whenever a non-digit arrives (and at the end).

Basis: LeetCode 339, Nested List Weight Sum, with the nested list given as a string.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(text):
    nested = json.loads(text)  # parse the lists
    return depth_sum(nested, 1)
def depth_sum(items, depth):
    total = 0
    for item in items:
        if isinstance(item, int):
            total += item * depth
        else:
            total += depth_sum(item, depth + 1)
    return total
```

TIME	SPACE
O(n)	O(depth)

WHY IT IS SLOW

It builds the whole nested structure and recurses as deep as the nesting, so memory and the call stack grow with depth. A single left-to-right scan with a bracket counter needs O(1) extra space.

OPTIMAL SOLUTION · Single pass over the string with a depth counter

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 When does the recursive (DFS) version become unsafe?

Python's default recursion limit is about 1000 frames, so an input nested 10,000 levels deep raises RecursionError. DFS also uses O(depth) call-stack space. The single-pass scanner uses O(1) and BFS uses O(widest level); both avoid the problem. In Java the same limit appears as StackOverflowError.

FOLLOW-UP 2 What is the inverse version (deeper integers weigh less)?

That is LeetCode 364. Do a level-by-level pass that keeps a running sum of everything seen so far and add it to the total once per level: total += running_sum on each level. Shallow integers are added many times, deep ones once, which equals weight (maxDepth + 1 - depth).

FOLLOW-UP 3 The inverse version (shallow integers weigh less, LeetCode 364).

Process level by level and keep a running sum of every integer seen so far; add the running sum to the total once per level that holds an integer or lies above one (trim trailing levels that contain only empty lists). Shallow numbers are added at every deeper level, which equals the weight maxDepth - depth + 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def with_depths(nested):
    # every integer with its depth (top level = 1),
    # using a stack so deep input cannot overflow
    stack = [(nested, 1)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            yield item, depth - 1


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
def weight_sum(nested):
    # deeper integers weigh more (LeetCode 339)
    return sum(v * d for v, d in with_depths(nested))


# FOLLOW-UP 2, 3: What is the inverse version (deeper
# integers weigh less)?
def inverse_weight_sum(nested):
    # deeper integers weigh less (LeetCode 364)
    pairs = list(with_depths(nested))
    if not pairs:
        return 0
    deepest = max(d for _, d in pairs)
    # weight = deepest + 1 - depth
    return sum(v * (deepest + 1 - d) for v, d in pairs)


# BASE QUESTION (optimal): Single pass over the string
# with a depth counter
def weight_sum_string(s):
    # one pass over the text, O(1) extra space
    depth = total = 0
    num, sign, have = 0, 1, False
    # a closing sentinel flushes a trailing number
    for ch in s + "]":
        if ch.isdigit():
            num, have = num * 10 + int(ch), True
            continue
        # any other character ends the number being read
        if have:
            total += sign * num * depth
            num, sign, have = 0, 1, False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total


if __name__ == "__main__":
    data = [3, 8, [2, 14], [2, [91]]]
    # 3 + 8 + 2*2 + 14*2 + 2*2 + 91*3 = 320
    assert weight_sum(data) == 320
    text = "[3, 8, [2, 14], [2, [91]]]"
    assert weight_sum_string(text) == 320
    assert weight_sum_string("[2, [91]]") == 2 + 91 * 2
    # inverse weights: [[1,1],2,[1,1]] -> 8
    assert inverse_weight_sum([[1, 1], 2, [1, 1]]) == 8
    assert inverse_weight_sum([1, [4, [6]]]) == 17
    # empty lists add nothing
    assert weight_sum([[], [[]]]) == 0
    # 20000 levels deep is still fine
    deep = []
    for _ in range(20000):
        deep = [deep, 1]
    assert weight_sum(deep) > 0
```

NOTES FROM CANDIDATES

The easy-to-miss bug is leaving the last buffered number uncommitted when the string ends. Manually tracing the small nested case exposed this during the interview, and fixing it proactively was part of the signal.
DFS visits each nested value once in O(n) time and uses O(d) call-stack space at maximum depth d; BFS is also O(n) time with a frontier proportional to the widest level. In the string parser, track the sign and whether a number is currently buffered, flush only when that flag is set, and flush before decrementing depth on].

PREPARATION

Write DFS and BFS versions over a parsed nested-list interface and explain when recursion depth becomes unsafe.
Build the string parser as a tiny state machine, including multi-digit values and the end-of-input flush.
Dry-run [2, [91]] aloud while tracking depth, buffered number, and accumulated sum.

MEDIUM | 1Point3Acres | onsite-coding | 10 of 33

Template Variable Expansion with Follow-Ups

ASKED 2026-08-14 | 57 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 210/224

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

QUESTION

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string.
Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values.
Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

APPROACH

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

HOW TO THINK ABOUT IT

Agree the grammar first: %NAME% is a placeholder, %% is a literal percent sign, an unmatched % is an error and an unknown name is an error. Values may contain placeholders themselves, so expand a value by expanding its own placeholders first. A cycle (A uses B, B uses A) would recurse forever, so detect it with a visiting set or, equivalently, with indegrees and a queue.

Basis: Written for this question; resolving values that contain other placeholders is a dependency graph problem (as in LeetCode 210, Course Schedule II).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def expand(text, depth=0):
    if depth > LIMIT:
        raise Error("too deep: cycle?")
    out = ""
    for token in tokens(text):
        if token == "%%":
            out += "%"
        elif is_key(token):
            value = values[name_of(token)]
            out += expand(value, depth + 1)
        else:
            out += token
    return out
```

BRUTE FORCE

TIME	SPACE
exponential in the worst ... exponential in the worst case	**O(depth)**

WHY IT IS SLOW

Re-expands shared values every time and finds cycles only by running out of depth. Memoising each value and keeping a visiting set expands each key once and reports a cycle at once.

OPTIMAL SOLUTION · Depth-first expansion with memo and a visiting set

TIME	SPACE
O(total length of the expanded text + keys)	**O(keys)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Detect cycles before expanding.

Model each key as a vertex with an edge to every key its value mentions, count indegrees and run Kahn's algorithm: if fewer vertices than keys come out, some keys sit on a cycle and cannot be resolved. This reports the problem up front instead of failing halfway through a long document.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict, deque


def tokens(s):
    # split text into ("text", char) and ("var", name)
    i = 0
    while i < len(s):
        if s[i] != "%":
            yield "text", s[i]
            i += 1
        elif s[i:i + 2] == "%%":
            # %% is a literal percent sign
            yield "text", "%"
            i += 2
        else:
            j = s.find("%", i + 1)
            if j < 0:
                raise ValueError("unmatched %")
            yield "var", s[i + 1:j]
            i = j + 1


# BASE QUESTION (optimal): Depth-first expansion with memo
# and a visiting set
def expand_template(text, mapping):
    done, visiting = {}, set()

    def resolve(name):
        # values may hold placeholders: expand them first
        if name in done:
```

```
            return done[name]
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        return "".join(resolve(x) if kind == "var" else x
                       for kind, x in tokens(s))

    return scan(text)
```

```
# FOLLOW-UP 1: Detect cycles before expanding.
def has_cycle(mapping):
    # Kahn's algorithm over key -> keys it mentions
    deps = {}
    for k, v in mapping.items():
        deps[k] = {x for kind, x in tokens(v)
                   if kind == "var"}
    users, indeg = defaultdict(list), {}
    for k, d in deps.items():
        known = d & mapping.keys()
        indeg[k] = len(known)
        for x in known:
            users[x].append(k)
    queue = deque(k for k, n in indeg.items() if n == 0)
    resolved = 0
    while queue:
        k = queue.popleft()
        resolved += 1
        for u in users[k]:
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    # keys left over sit on a cycle
    return resolved != len(mapping)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    assert expand_template(text, m) == (
        "I am admin My home is /admin/home")
    sure = expand_template("100%% sure", {})
    assert sure == "100% sure"
    assert not has_cycle(m)
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    for bad, error in (("a % b", ValueError),
                       ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
        except error:
            pass
```

```
    try:
        expand_template("%A%", loop)
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
```

NOTES FROM CANDIDATES

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

PREPARATION

Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

MEDIUM | 1Point3Acres | onsite-coding | 11 of 33

Target Expressions from Ordered Digits

ASKED 2026-08-13 | 58 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 36.5 | 30 DAYS 0 | PRIORITY 102/224

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 282 Expression Add Operators

Frequency: Single report | Last asked: 2026-08-13 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/target-expressions-ordered-digits

QUESTION

EXAMPLES

```
For digits [2, 3, 4] and target 14, valid outputs
    include:
2 * (3 + 4)
2 + 3 * 4
```

APPROACH

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether digits may be concatenated into multi-digit numbers, which operators are allowed, whether parentheses are free, whether the digit order is fixed, and whether to return all expressions or a boolean.

BRUTE FORCE TO SAY OUT LOUD

INTERVIEW-DAY PLAN

Enumerate every operator placement and every parenthesization for the three or four digits and evaluate each. That is fine for a handful of digits and is what the first stage asks for.

OPTIMAL IDEA

Interval DP: for each interval of digits compute a map from value to the expressions that achieve it by splitting at every point and combining left and right with plus, minus and times. Memoize by interval. If only reachability is needed store just the set of values per interval, which is much smaller than the strings.

DRY-RUN THESE CASES

[2, 3, 4] with target 14 gives 2*(3+4) and 2+3*4. A target that is unreachable. A single digit. Zeros.

LIKELY FOLLOW-UPS

N digits (the DP above), division with fractions, or concatenation.

PITFALLS

Missing parentheses in the output string. Evaluating with floats.

HOW TO THINK ABOUT IT

Any expression over a run of digits is two smaller expressions joined by an operator at some split point, so the set of values for an interval is built from the sets of its two halves. Compute values bottom-up per interval and only build text for the target. Be explicit about parentheses: they come from the split structure, so every split is a candidate.

Basis: Related to LeetCode 282, Expression Add Operators, but with parentheses allowed and digits not concatenated.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def values(digits):
    if len(digits) == 1:
        return {digits[0]}
    result = set()
    for cut in range(1, len(digits)):
        for a in values(digits[:cut]):
            for b in values(digits[cut:]):
                result |= {a + b, a - b, a * b}
    return result
reachable = target in values(all_digits)
```

TIME	SPACE
exponential (Catalan(n) *... exponential (Catalan(n) * 3^(n-1) expressions)	**O(n)** recursion plus the value set

WHY IT IS SLOW

The same sub-interval of digits is recomputed from scratch for every split above it, giving Catalan-number growth. Remembering the values per interval (memoisation) makes it polynomial in the number of distinct values.

OPTIMAL SOLUTION · Interval DP: all expressions for digits[l:r]

TIME	SPACE
exponential in the number... exponential in the number of results; polynomial in the number of distinct values	**O(distinct values per interval)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you handle N digits where listing every expression is too much?

Agree on the contract: if you only need to know whether a target is reachable, store just the set of values per interval (a boolean DP over (l, r, value)), which is far smaller than the set of strings. If you need one witness, store one expression string per value. Values are bounded by the digits, so the table stays practical for moderate N.

FOLLOW-UP 2 What if concatenating digits (23) or division is allowed?

Concatenation adds base cases: for every l and r the number int(digits[l:r]) is also an atom of the interval. Division adds a fourth operator with a zero check and, if exactness matters, Fraction values instead of floats.

FOLLOW-UP 3 Only a yes/no answer is needed for many digits.

Store just the set of reachable values per interval (as target_reachable does) instead of the expression strings; the strings blow up long before the value sets do. Build the expression only for the winning target by remembering one split per value.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict


def build(digits):
    # table[(l, r)] = {value: {(text, precedence)}}
    table = {}

    def solve(l, r):
        if (l, r) in table:
            return table[(l, r)]
        res = defaultdict(set)
        if r - l == 1:
            # a single digit is an atom (precedence 3)
            res[digits[l]].add((str(digits[l]), 3))
        # try every split point
        for m in range(l + 1, r):
            for lv, ls in solve(l, m).items():
                for rv, rs in solve(m, r).items():
                    combine(res, lv, ls, rv, rs)
        table[(l, r)] = res
```

```
        return res

    return solve(0, len(digits))


def combine(res, lv, ls, rv, rs):
    ops = (("+", 1, lv + rv), ("-", 1, lv - rv),
           ("*", 2, lv * rv))
    for lt, lp in ls:
        for rt, rp in rs:
            for op, prec, val in ops:
                # brackets only where precedence needs
                left = f"({lt})" if lp < prec else lt
                need = rp < prec
                if rp == prec and op == "-":
                    need = True
                right = f"({rt})" if need else rt
                res[val].add((left + op + right, prec))


# BASE QUESTION (optimal): Interval DP: all expressions
# for digits[l:r]
def expressions_for_target(digits, target):
    found = build(digits).get(target, ())
    return sorted({text for text, _ in found})
```

```
# FOLLOW-UP 3: Only a yes/no answer is needed for many
# digits.
def target_reachable(digits, target):
    # yes/no only: keep just the set of values per range
    values = {}

    def solve(l, r):
        if (l, r) in values:
            return values[(l, r)]
        out = {digits[l]} if r - l == 1 else set()
        for m in range(l + 1, r):
            for a in solve(l, m):
                for b in solve(m, r):
                    out |= {a + b, a - b, a * b}
        values[(l, r)] = out
        return out

    return target in solve(0, len(digits))


if __name__ == "__main__":
    ans = expressions_for_target([2, 3, 4], 14)
    assert "2*(3+4)" in ans and "2+3*4" in ans
    assert target_reachable([2, 3, 4], 14)
    assert not target_reachable([2, 3, 4], 100)
    # subtraction needs brackets on the right
    sub = expressions_for_target([5, 3, 1], 3)
    assert "5-(3-1)" in sub
    assert target_reachable([1] * 8, 8)
```

NOTES FROM CANDIDATES

The base case is small enough for direct enumeration, but a complicated ad hoc implementation becomes difficult to extend. Keep expression construction separate from numeric evaluation while negotiating the N-digit contract.
If the agreed contract uses every digit exactly once in order, memoize each interval [l, r) as a map from attainable numeric values to the expression strings producing them. For every split point, combine left and right entries with + and *, wrapping composite children as needed; the full-interval target bucket is the answer. Because every distinct expression must be retained, both time and storage are output-sensitive and can grow exponentially even though interval states are memoized.

PREPARATION

Enumerate operator placements and valid parenthesizations for three or four ordered digits.
Explore memoized dynamic-programming state definitions for the N-digit follow-up and be ready to justify one before coding.
Explain how duplicate expressions or duplicate values would be handled without discarding distinct valid output forms.

MEDIUM | 1Point3Acres | phone-screen | 12 of 33

Progressive Array Partitioning

ASKED 2026-08-11 | 60 DAYS AGO | LAST 90 DAYS

STUDY: TALK | NO LEETCODE FREQUENCY | PRIORITY 194/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-08-11 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/array-partition-unique-frequencies-straights

QUESTION

EXAMPLES

Stage 1

```
[1, 2, 3, 4, 2, 3, 3, 4, 4, 4] -> true because the
    four value-groups have different sizes.
[1, 2, 3] -> false because every value-group has size
    one.
```

Stage 2

```
[1, 2, 2, 3, 3, 4, 4, 5, 5, 6] -> true.
[4, 5, 6, 7, 7, 8, 9] -> false.
```

APPROACH

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what the groups are and the rule for each stage. From the examples, the first stage looks like: true when the counts of all distinct values are different from each other (1:1, 2:2, 3:3, 4:4 is true; all counts 1 is false). The second stage is not clear from the examples alone, so ask for its exact rule and test it on both examples before coding.

BRUTE FORCE TO SAY OUT LOUD

Try every way to group the elements and test the rule. Exponential.

OPTIMAL IDEA

For stage one: count each value with a hash map, then check that the multiset of counts has no repeats (a set of counts has the same size as the number of values). O(n). Keep the data model (a counter) easy to adapt, because the second stage arrives only after the first is written.

DRY-RUN THESE CASES

The four examples in the statement, an empty array, a single value, many values with equal counts.

LIKELY FOLLOW-UPS

Stage two, which changes the grouping rule; keep helper functions small so it is a local change.

PITFALLS

Coding stage two from a guess. Not reconciling the rule with both examples.

HOW TO THINK ABOUT IT

The statement arrives in stages, so keep the data model easy to adapt: count the values first, because any rule about 'groups of equal values' is a rule about the counts. Stage 1 is then 'are all counts different', which is a set-size comparison. Before stage 2, reconcile the new rule with every example; here a natural guess (LeetCode 659) contradicts the second example, which is exactly the situation where you ask.

Basis: Written. Stage 1 is inferred from the examples: true when every distinct value occurs a different number of times. The Stage 2 rule is NOT recoverable from the two examples, so only a candidate reading is shown; ask the interviewer for the exact rule and test it against both examples before coding.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(a):
    counts = Counter(a)
    for x, y in combinations(counts, 2):
        if counts[x] == counts[y]:
            return False
    return True
```

TIME	SPACE
O(d^2) where d is the number of distinct values	O(d)

WHY IT IS SLOW

O(d^2) pair checks for d distinct values. Comparing the number of distinct counts with the number of distinct values (a set) is O(n).

OPTIMAL SOLUTION · Counter, then compare the set of counts with the number of values

TIME	SPACE
O(n)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Stage 2 arrives. What do you do?

Do not guess. Write down what the second example pair implies, and check any candidate rule against BOTH. The common reading 'split into consecutive runs of length at least 3' gives true for [4, 5, 6, 7, 7, 8, 9], but the statement says false, so there is an extra condition (such as run lengths or a count rule) to ask about.

FOLLOW-UP 2 Keep the code adaptable between stages.

Put the counting in one helper and each stage's rule in its own small function that takes the counts. Stage 2 then replaces a function instead of rewriting the pipeline.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import Counter


def counts_of(nums):
    # every rule about groups of equal values is a
    # rule about these counts, so compute them once
    return Counter(nums)


# BASE QUESTION (optimal): Counter, then compare the set
# of counts with the number of ...
def stage1(counts):
    # true when every value occurs a different number of
```

```
    # times: the counts have no repeats
    return len(set(counts.values())) == len(counts)


# FOLLOW-UP 1: Stage 2 arrives. What do you do?
def stage2_candidate(nums, min_len=3):
    # ONE reading of stage 2 (LeetCode 659): split the
    # array into runs of consecutive values, each at
    # least min_len long. Check it against BOTH examples.
    left, ends = Counter(nums), Counter()
    for x in nums:
        if left[x] == 0:
            continue
        left[x] -= 1
        if ends[x - 1] > 0:
            # extend a run that ended at x - 1
            ends[x - 1] -= 1
            ends[x] += 1
        elif left[x + 1] > 0 and left[x + 2] > 0:
            # start a new run x, x + 1, x + 2
            left[x + 1] -= 1
            left[x + 2] -= 1
            ends[x + 2] += 1
        else:
            return False
    return True


# each stage is one small function, so a new rule
# replaces one entry instead of the whole pipeline
RULES = {1: lambda nums: stage1(counts_of(nums)),
         2: stage2_candidate}
```

```
def check(stage, nums):
    return RULES[stage](nums)


if __name__ == "__main__":
    # stage 1 examples
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])
    # stage 2 example that the candidate gets right
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])
    # the second stage-2 example says False, but this
    # reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True
```

NOTES FROM CANDIDATES

The interviewer presents Stage 2 only after the first implementation, so keep the initial data model easy to adapt.
Before coding the straight variant, reconcile the length rule with both examples; otherwise a superficially valid decomposition can contradict the expected result.
Evaluation emphasized the efficiency of the chosen data structures, not only whether the sample cases passed.
Solution skeleton
Stage 1: count each distinct value, then verify that the frequency values are themselves unique. A hash map plus a set gives O(n) time and O(u) space for u distinct values.
Stage 2: first pin down a group size k. For the fixed-size interpretation, reject when n % k != 0, process keys in ascending order, and whenever the smallest remaining key x has multiplicity c, remove c copies from every key in [x, x + k). Any missing count makes the partition impossible. Sorting the u keys gives O(n + u log u) time and O(u) space.
Do not let length-one groups make every input trivially valid unless the interviewer explicitly allows them; the negative example requires a stronger group-size contract.

PREPARATION

Implement both stages against repeated values, singletons, empty input, and inputs with many duplicate counts.
Practice stating the partition invariant and asking about run length, duplicate consumption, and whether every element must be used exactly once before writing code.

MEDIUM | 1Point3Acres | phone-screen | 13 of 33

2D Points Connected Components by Radius

ASKED 2026-08-04 | 67 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 40.7 | 30 DAYS 0 | PRIORITY 90/224

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #547 Number of Provinces

LeetCode match: LC 547 Number of Provinces (related)

Frequency: Low | Last asked: 2026-08-04 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/points-connected-components-by-radius

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

QUESTION

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

APPROACH

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

HOW TO THINK ABOUT IT

Two points are neighbours when their squared distance is at most r squared (no square roots needed), and connectivity is transitive, so this is counting connected components: union-find or a DFS over the neighbour relation. Checking every pair is O(n^2); bucketing points into cells of side r means a point only needs to be compared with the 3 x 3 cells around it.

Basis: LeetCode 547, Number of Provinces, where two points are linked when their distance is at most r.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(points, r):
    graph = defaultdict(list)
    ids = range(len(points))
    for i, j in combinations(ids, 2):
        if dist2(points[i], points[j]) <= r * r:
            graph[i].append(j)
            graph[j].append(i)
    seen, clusters = set(), 0
    for i in range(len(points)):
        if i not in seen:
            clusters += 1
            dfs(graph, i, seen)
    return clusters
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every pair is compared, O(n^2). Putting points in cells of side r means each point is compared only with the 3 x 3 cells around it.

OPTIMAL SOLUTION · Union-find with a spatial grid

TIME	SPACE
O(n) about when points are spread out (O(n^2) if they all share a cell)	O(n)

OPTIMAL SOLUTION · Union-find with a spatial grid

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why compare squared distances?

sqrt is slow and inexact; dist <= r is the same as dist^2 <= r^2 for non-negative numbers, and with integer coordinates the comparison is exact. It also avoids floating-point surprises right at the boundary.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict


class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        # path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# FOLLOW-UP 1: Why compare squared distances?
def near(p, q, r):
    # squared distance: no square root, exact for ints
    dx, dy = p[0] - q[0], p[1] - q[1]
    return dx * dx + dy * dy <= r * r


# BASELINE: compare every pair
def count_clusters_pairs(points, r):
    # the accepted baseline: compare every pair, O(n^2)
    uf = UnionFind(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if near(points[i], points[j], r):
                uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


# BASE QUESTION (optimal): Union-find with a spatial grid
def count_clusters(points, r):
    # grid of cells r wide: only 9 cells need a look
    size = r if r > 0 else 1
    cells = defaultdict(list)
    for i, (x, y) in enumerate(points):
        cells[(x // size, y // size)].append(i)
    uf = UnionFind(len(points))
    for i, (x, y) in enumerate(points):
```

```
        cx, cy = x // size, y // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                cell = cells.get((cx + dx, cy + dy), ())
                for j in cell:
                    q = points[j]
```

```
                    if j > i and near((x, y), q, r):
                        uf.union(i, j)
    return len({uf.find(i) for i in range(len(points))})


if __name__ == "__main__":
    # three points joined only through the middle one
    line = [(0, 0), (3, 0), (6, 0)]
    assert count_clusters(line, 3) == 1
    assert count_clusters_pairs(line, 3) == 1
    # the far point stays alone
    far = [(0, 0), (3, 0), (7, 0)]
    assert count_clusters(far, 3) == 2
    # radius 0 only joins identical points
    same = [(1, 1), (1, 1), (2, 2)]
    assert count_clusters(same, 0) == 2
    assert count_clusters([], 5) == 0
    # both methods agree on a scatter of points
    pts = [(i * 7 % 13, i * 5 % 11) for i in range(30)]
    for r in (0, 1, 2, 3, 5):
        assert count_clusters(pts, r) == (
            count_clusters_pairs(pts, r))
```

NOTES FROM CANDIDATES

DFS, BFS, or Union-Find all fit the core connected-components model.
For many points, discuss how to avoid checking every pair by narrowing neighbor candidates spatially; in this round, the interviewer still preferred the brute-force implementation path.
Clarify squared-distance comparison to avoid unnecessary square roots.

PREPARATION

Implement connected components once with DFS/BFS and once with Union-Find.
Practice stating the O(n^2) brute-force baseline clearly before proposing spatial indexing.
Dry-run a case where three points form one cluster only through transitive links.

MEDIUM | IN APP LC 295 | DESIGN-STYLE (skippable) | 1Point3Acres | tech-screen | 14 of 33

Streaming Median (LC 295) with Follow-ups

ASKED 2026-08-03 | 68 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | **ALL TIME 50.4** | **30 DAYS 0** | **PRIORITY 68/224**

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #295)

LeetCode match: LC 295 Find Median from Data Stream

Frequency: Single report | Last asked: 2026-08-03 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/streaming-median-with-follow-ups

QUESTION

A near-canonical LC 295 streaming-median implementation using two heaps, followed by two design extensions: exploit a very small value domain, then support an arbitrary k-th-smallest query instead of only the median.SWEheapstreamingmediumFrequencySingle reportLast asked2026-08-03Stagetech-screen

APPROACH

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether values are integers, the range of values, how often median queries happen compared with inserts, and what to return for an even count (average of two middles).

BRUTE FORCE TO SAY OUT LOUD

Keep a sorted list and insert with bisect; median is the middle. O(n) per insert due to shifting.

OPTIMAL IDEA

Two heaps: a max-heap for the lower half and a min-heap for the upper half, sizes within one. Insert by pushing through one heap into the other and rebalancing. Median is the top of the larger heap or the average of both tops. O(log n) insert, O(1) median.

DRY-RUN THESE CASES

One element, two elements, duplicates, all equal, alternating very large and very small values.

LIKELY FOLLOW-UPS

Values in a small range (counting array), arbitrary k-th smallest (Fenwick tree or order-statistic tree), or too much data (quantile sketch).

PITFALLS

Rebalancing after the wrong heap push. Integer division when the median should be fractional.

HOW TO THINK ABOUT IT

Keep the data split at the median: a max-heap for the smaller half and a min-heap for the larger half, with sizes equal or differing by one. Then the median is read from the tops. Insert by pushing through one heap into the other to keep every smaller value on the left, and rebalance the sizes.

Basis: LeetCode 295, Find Median from Data Stream.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
nums = []  # always kept sorted
def add(x):
    insort(nums, x)
def median():
    n = len(nums)
    if n % 2:
        return nums[n // 2]
    return (nums[n // 2 - 1] + nums[n // 2]) / 2
```

TIME	SPACE
O(n) per insert (list shift), O(1) median	**O(n)**

WHY IT IS SLOW

Each insert shifts up to n elements, O(n) per add (fast in practice, but linear). Two heaps keep the median available with O(log n) per add and O(1) per query.

OPTIMAL SOLUTION · Two heaps

TIME	SPACE
O(log n) add, O(1) median	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?

Yes: a counting array gives O(1) inserts, and the median needs one pass over at most V counts, so O(V) per query with V fixed. Keep a moving pointer to the median bucket to make queries amortized O(1).

FOLLOW-UP 2 Now support an arbitrary k-th smallest, not only the median.

Two heaps only work for one fixed rank. Use an order-statistics structure: a Fenwick tree over (compressed) values, or a balanced BST augmented with subtree sizes. Both give O(log n) add and O(log n) k-th, and the Fenwick tree is far shorter to write in an interview.

FOLLOW-UP 3 What if the data stream is too large to store?

Exact medians need memory proportional to the data, so use a quantile sketch (t-digest, GK, KLL). They answer any quantile within a chosen error bound in O(1/epsilon) memory.

FOLLOW-UP 4 The median of only the last K values.

Keep the window in a deque (to know what expires) and a sorted list (to read the middle); insert and remove with bisect. O(K) per update because of list shifting; two heaps with lazy deletion give O(log K).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from bisect import bisect_left, insort


# BASE QUESTION (optimal): Two heaps
class MedianFinder:
    def __init__(self):
        # low: max-heap (stored negated); high: min-heap
        self.low, self.high = [], []

    def add(self, x):
        # push through low so the halves stay ordered
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        # keep low the same size as high, or one bigger
        if len(self.high) > len(self.low):
            top = heapq.heappop(self.high)
            heapq.heappush(self.low, -top)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# FOLLOW-UP 1: Almost every value is in a small range (say
# 0 to 100). Can you do ...
class CountingMedian:
    """Follow-up: values are whole numbers in 0..maxv."""

    def __init__(self, maxv):
        self.counts = [0] * (maxv + 1)
        self.n = 0

    def add(self, x):
        self.counts[x] += 1
        self.n += 1

    def kth(self, k):
        # k-th smallest (1-based): one pass over the
        # counts
        seen = 0
        for value, c in enumerate(self.counts):
            seen += c
```

```
            if seen >= k:
                return value

    def median(self):
        lo = self.kth((self.n + 1) // 2)
        hi = self.kth(self.n // 2 + 1)
        return (lo + hi) / 2


# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not
# only the median.
class Fenwick:
```

```
    """Follow-up: any k-th smallest in O(log V)."""

    def __init__(self, size):
        self.size, self.tree = size, [0] * (size + 1)

    def add(self, value):
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i

    def kth(self, k):
        # walk down the tree to the k-th smallest value
        pos, step = 0, 1 << self.size.bit_length()
        while step:
            nxt = pos + step
            if nxt <= self.size and self.tree[nxt] < k:
                pos = nxt
                k -= self.tree[nxt]
            step >>= 1
        return pos


# FOLLOW-UP 4: The median of only the last K values.
class WindowMedian:
    """Follow-up: median of only the last k values."""

    def __init__(self, k):
        self.k, self.order, self.sorted = k, [], []

    def add(self, x):
        self.order.append(x)
        insort(self.sorted, x)
```

```
        if len(self.order) > self.k:
            old = self.order.pop(0)
            del self.sorted[bisect_left(self.sorted, old)]

    def median(self):
        s, n = self.sorted, len(self.sorted)
        return (s[(n - 1) // 2] + s[n // 2]) / 2


if __name__ == "__main__":
    m = MedianFinder()
    for x in [5, 2, 8]:
        m.add(x)
    assert m.median() == 5
    m.add(1)
    assert m.median() == 3.5
    c = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        c.add(x)
    assert c.median() == 3.5
    f = Fenwick(100)
    for x in [5, 2, 8, 1]:
        f.add(x)
    ranks = [f.kth(k) for k in (1, 2, 3, 4)]
    assert ranks == [1, 2, 5, 8]
    w = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        w.add(x)
    # window is [1, 5, 7]
    assert w.median() == 5
```

NOTES FROM CANDIDATES

Keep the lower half in a max-heap and the upper half in a min-heap. Maintain a size difference of at most one and keep every lower-half value no greater than every upper-half value.
For an even-sized stream, average the two heap roots.
Each insertion costs O(log n), a median query costs O(1), and the heaps use O(n) space.
For a small bounded domain, a frequency array gives O(1) updates and an O(U) order-statistic scan, where U is the domain size. For arbitrary k-th-smallest queries, use an order-statistic tree or a Fenwick/segment tree over a known or compressed domain for logarithmic update and selection.
The base problem required complete working code plus a dry run and complexity analysis.
The first follow-up stopped at design and pseudocode; the second was discussion-only because the round ran out of time.

PREPARATION

Implement the two-heap median structure and dry-run odd and even stream lengths.
Sketch a bounded-domain alternative and state its update and query costs.
Compare what must change when the requested order statistic is an arbitrary k rather than a fixed median.

MEDIUM | IN APP LC 394 | 1Point3Acres | onsite-coding | 15 of 33

★ TOP 75: rank 60 in the Medium and Hard priority list

★ Decode String

ASKED 2026-07-29 | 73 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 57.7 30 DAYS 0 PRIORITY 44/224

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: your app's interview approach (LeetCode #394)

LeetCode match: LC 394 Decode String

Frequency: Low | Last asked: 2026-07-29 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/decode-string

QUESTION

EXAMPLES

```
"3[a]2[bc]" -> "aaabcbc".
"2[abc]3[cd]ef" -> "abcabccdcdcdef".
"3[a2[c]]" -> "accaccacc".
```

APPROACH

Write the iterative-stack version from memory in under 10 min. Practice the postfix-syntax variant (parse direction reversed). Drill the decode side together with the encode-shortest dual problem as a pair.

HOW TO THINK ABOUT IT

Nested repeats like 3[a2[c]] are a bracket structure, which means a stack: when you see '[' save what you have built and the repeat count; when you see ']' repeat the inner string and glue it to the saved prefix. Read counts as full numbers (more than one digit).

Basis: LeetCode 394, Decode String.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def parse():
    result = ""
    while pos < len(s) and s[pos] != "]":
        if s[pos].isdigit():
            n = read_number()
            pos += 1  # skip "["
            inner = parse()
            pos += 1  # skip "]"
            result += inner * n
        else:
            result += s[pos]
            pos += 1
    return result
```

TIME	SPACE
O(output length)	O(depth)

WHY IT IS SLOW

Time is proportional to the output, which is optimal; the cost is the recursion depth, equal to the nesting depth. A stack of (text so far, repeat count) avoids recursion.

OPTIMAL SOLUTION · Stack of (prefix, repeat) frames

TIME	SPACE
O(output length)	O(output + nesting depth)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you do it recursively, and what is the risk?

Parse with an index: read letters and digits, on '[' recurse to parse the inner string and return where it ended, on ']' return. It reads naturally but the call depth equals the nesting depth, which can overflow the stack for adversarial inputs. The stack version trades that for explicit memory.

FOLLOW-UP 2 What if the decoded string is enormous?

Do not materialize it. Return a generator, or compute the character at a given index by walking the structure with lengths (as in LeetCode 880, Decoded String at Index), using index modulo the segment length at each level.

FOLLOW-UP 3 The decoded string would be enormous.

Compute its length (or a given character) from the stack without building it: keep (length so far, repeat count) frames and combine as prev + inner * times. The length can be computed for strings that could never fit in memory.

FOLLOW-UP 4 Return the character at position k only.

Walk the structure with lengths: at each repeat reduce k modulo the inner length to find where in the inner string it falls (LeetCode 880 style).

FOLLOW-UP 5 Invalid input (missing bracket, count with no bracket).

Detect an unmatched ']' (empty stack), a '[' left at the end, and digits not followed by '['. Decide whether to raise or to treat a malformed part as literal text.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Stack of (prefix, repeat)
# frames
def decode(s):
    # stack of (text so far, repeat count) for each open [
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            # remember what we have, start the inner text
            stack.append((cur, num))
            cur, num = [], 0
        elif ch == "]":
            prev, times = stack.pop()
            prev.append("".join(cur) * times)
            cur = prev
        else:
            cur.append(ch)
    return "".join(cur)


# NESTED FORM (the recursive version)
def parse(s):
    # nested form: items are text or (times, items)
    stack, cur, num = [], [], 0
    for ch in s:
        if ch.isdigit():
            num = num * 10 + int(ch)
        elif ch == "[":
            if num == 0:
                raise ValueError("[ needs a count")
            stack.append((cur, num))
            cur, num = [], 0
```

```
        elif ch == "]":
            if not stack:
                raise ValueError("unmatched ]")
            prev, times = stack.pop()
            prev.append((times, cur))
            cur = prev
        else:
            if num:
                raise ValueError("count without [")
            cur.append(ch)
    if stack:
        raise ValueError("missing ]")
```

```
    return cur


# FOLLOW-UP 3: The decoded string would be enormous.
def length(items):
    # decoded length without building the string
    total = 0
    for it in items:
        if isinstance(it, str):
            total += 1
        else:
            total += it[0] * length(it[1])
    return total


# FOLLOW-UP 4: Return the character at position k only.
def char_at(items, k):
    # the k-th decoded character (0-based), LeetCode 880
    # style
    for it in items:
        if isinstance(it, str):
            if k == 0:
                return it
            k -= 1
        else:
            times, inner = it
            inner_len = length(inner)
            if k < times * inner_len:
                # which copy does not matter: use k modulo
                return char_at(inner, k % inner_len)
            k -= times * inner_len
    return None


if __name__ == "__main__":
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    items = parse("3[a2[c]]")
    assert length(items) == 9
    assert [char_at(items, i) for i in range(9)] == list(
        decode("3[a2[c]]"))
    # a huge result is measured, never built
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3
```

```
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass
```

NOTES FROM CANDIDATES

Google variants include alternative syntax (group){k} (postfix repeat), nested grouping with curly braces, and escape characters.
A scale follow-up asks how the interface and memory strategy should change when the encoded string is extremely long.
Recursive solution is cleaner but the iterative stack is more robust to bizarre input.

PREPARATION

Write the iterative-stack version from memory in under 10 min.
Practice the postfix-syntax variant (parse direction reversed).
Drill the decode side together with the encode-shortest dual problem as a pair.

ALSO REPORTED · 1 more

String Decompression with Nested Counts | 1Point3Acres

https://www.1point3acres.com/interview/problems/company/google/string-decompression-nested-counts

Recursion with an index pointer is the cleanest implementation: each call returns the decoded substring of a single (...){k} block plus literals up to its terminator.
An iterative stack-based approach also works: push (StringBuilder, repeatCount) on (, pop and append on){k}.
The 2-digit k matters only for parsing — read until non-digit.
Watch out for literals between two groups: a(b){2}c(d){3} -> abbcddd.

Preparation: Drill the canonical k[group] decode-string problem to muscle memory; the input grammar there is k[group] instead of (group){k}, but the algorithm is identical.
Practice both recursive and iterative versions — the iterative stack version generalizes more cleanly if the interviewer asks about iterative streaming.
Test with: deeply nested groups, groups containing only literals, two-digit counts ({12}).

MEDIUM | DESIGN-STYLE (skippable) | 1Point3Acres | onsite-coding | 16 of 33

Batch and Streaming Log Deduplication

ASKED 2026-07-23 | 79 DAYS AGO | LAST 90 DAYS

STUDY: READ **NO LEETCODE FREQUENCY** **PRIORITY 222/224**

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

This question has several parts, so each part has its own answer below.

Frequency: Single report | Last asked: 2026-07-23 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/batch-streaming-log-deduplication

QUESTION

Process **`(timestamp, message)`** logs by deduplicating on message and sorting retained entries by timestamp, first keeping the first-seen entry and then the latest timestamp. The streaming extension adds **`add(timestamp, message)`** and **`get_next()`**, requiring a map plus min-heap, lazy deletion, and a discussion of watermarks and bounded state.SWEdedupstreamingheapsortingmediumFrequencySingle reportLast asked2026-07-23Stageonsite-coding

APPROACH

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask which entry to keep in each policy (first seen or latest timestamp), how ties on timestamp break, whether output is sorted by timestamp, and how the streaming interface returns results (get_next).

BRUTE FORCE TO SAY OUT LOUD

Sort all logs and dedupe with a set. Correct for the batch version.

OPTIMAL IDEA

Batch: keep a dict message to chosen entry (keep first-seen, or replace if the new timestamp is later) and sort the values by (timestamp, message). Streaming: a dict for membership plus a min-heap of (timestamp, message); on update of a message push a new entry and mark the old one stale (lazy deletion), and get_next pops until it finds a non-stale one. Amortized O(log n).

DRY-RUN THESE CASES

The same message twice at different times, equal timestamps, an empty stream, get_next on an empty queue.

LIKELY FOLLOW-UPS

Watermarks for late data, bounded memory with eviction by age.

PITFALLS

Returning a stale heap entry. Forgetting deterministic tie-breaking.

HOW TO THINK ABOUT IT

Dedup is a dictionary keyed by the message. Sorting is only for the output order. For a stream, a heap gives the smallest timestamp quickly, but when an entry is replaced you cannot delete from the middle of a heap, so leave the stale entry and skip it when it surfaces (lazy deletion).

HOW TO THINK ABOUT IT

Basis: Written. Assumed rules (confirm them): dedupe on the message; policy 'first' keeps the earliest in input order, policy 'latest' keeps the largest timestamp; output sorted by (timestamp, message); get_next returns the retained entry with the smallest timestamp and removes it.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(logs, keep_first=True):
    kept = []
    for entry in logs:
        same = [x for x in logs
                if x[1] == entry[1]]
        pick = max(same)
        if keep_first:
            pick = same[0]
        if entry == pick:
            kept.append(entry)
    return sorted(kept)  # by timestamp
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Each line scans the others: O(n^2). A dictionary from message to the kept entry is one pass, and a heap with lazy deletion serves the streaming version.

OPTIMAL SOLUTION · Dictionary on the message, then sort

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

OPTIMAL SOLUTION · Streaming: map + min-heap with lazy deletion

TIME	SPACE
O(log n) amortized per add and get_next	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Late data: how do you decide when it is safe to emit?

Use a watermark: only release entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness). get_next then refuses to return anything newer than the watermark, and anything arriving older than it is dropped or sent to a late-data side output.

FOLLOW-UP 2 Memory must stay bounded.

Evict messages older than a retention window (a second heap or the same heap keyed on timestamp) and accept that duplicates arriving after eviction are no longer detected. **Cost:** O(log n) per eviction with a heap; memory O(entries inside the retention window).

FOLLOW-UP 3 Keep the entry with the most occurrences instead.

Store (count, timestamp) per message and update the count on every duplicate; the heap key changes only if you rank by count, so push a new entry when the rank changes and rely on lazy deletion again.

FOLLOW-UP 4 Only release entries once they are safe (late data may still arrive).

Add a watermark: release only entries whose timestamp is at most (the largest timestamp seen minus the allowed lateness), in order; anything older than the watermark that arrives later goes to a late-data path. The lazy-deletion heap pops stale entries as it goes.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq


# BASE QUESTION (optimal): Dictionary on the message, then
# sort
def dedupe(logs, policy="first"):
    # logs: (timestamp, message) pairs; keep one per
    # message
    kept = {}
    for ts, msg in logs:
        # "first" keeps the first seen, "latest" the
        # newest
        if msg not in kept or (
                policy == "latest" and ts > kept[msg]):
            kept[msg] = ts
    # retained entries sorted by time, then message
    return sorted((ts, msg) for msg, ts in kept.items())


# FOLLOW-UP 4: Only release entries once they are safe
# (late data may still arrive).
class StreamDedup:
    """add(ts, msg) and get_next() with a lazy heap."""

    def __init__(self, policy="first", lateness=0):
        self.policy, self.lateness = policy, lateness
        # message -> the timestamp currently kept
        self.kept = {}
        # (timestamp, message), may hold outdated entries
        self.heap = []
        self.max_ts = None

    def watermark(self):
        # everything at or below this is safe to release
        if self.max_ts is None:
            return None
        return self.max_ts - self.lateness

    def add(self, ts, msg):
        wm = self.watermark()
        # too late: older than the watermark, drop it
        if wm is not None and ts < wm:
            return False
        if self.max_ts is None or ts > self.max_ts:
            self.max_ts = ts

        old = self.kept.get(msg)
        if old is None or (
                self.policy == "latest" and ts > old):
            self.kept[msg] = ts
            heapq.heappush(self.heap, (ts, msg))
        return True

    def get_next(self):
        wm = self.watermark()
        while self.heap:
            ts, msg = self.heap[0]
            # lazy deletion: skip entries that were
            # replaced
            if self.kept.get(msg) != ts:
                heapq.heappop(self.heap)
                continue
            # not safe yet: later data may still arrive
            if ts > wm:
                return None
            heapq.heappop(self.heap)
            del self.kept[msg]
            return ts, msg
        return None


if __name__ == "__main__":
    logs = [(5, "b"), (1, "a"), (9, "a"), (3, "b")]
    assert dedupe(logs, "first") == [(1, "a"), (5, "b")]
    assert dedupe(logs, "latest") == [(5, "b"), (9, "a")]
    s = StreamDedup("first", lateness=2)
    for ts, msg in [(1, "a"), (3, "c"), (5, "b")]:
        assert s.add(ts, msg)
    # watermark is 5 - 2 = 3: release 1, then 3
    assert s.get_next() == (1, "a")
    assert s.get_next() == (3, "c")
    # 5 is still above the watermark
    assert s.get_next() is None
    # data older than the watermark is dropped
    assert not s.add(2, "late")
    # a "latest" policy replaces the kept entry
    t = StreamDedup("latest")
    t.add(1, "x")
    t.add(4, "x")
    assert t.get_next() == (4, "x")
    assert t.get_next() is None
```

NOTES FROM CANDIDATES

Empty input, identical entries, equal timestamps, out-of-order arrivals, updates, and removals all need explicit semantics.
For keep-latest streaming behavior, stale heap entries must be detected against the current timestamp stored for that message.
Arbitrarily late arrivals make globally final timestamp ordering impossible without a watermark or maximum-lateness guarantee.

PREPARATION

Implement both batch retention policies and write deterministic tie tests.
Build the streaming interface with a membership map, min-heap, and lazy deletion, then state the amortized operation costs.
Rehearse a bounded-memory design using expiry and heap rebuilding, plus a partitioning strategy that keeps each message on one shard.

MEDIUM | 1Point3Acres | onsite-coding | 17 of 33

Mahjong Winning Hand Detection (14 Tiles)

ASKED 2026-07-20 | 82 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 206/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-20 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/mahjong-winning-hand-check

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

APPROACH

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

HOW TO THINK ABOUT IT

A winning hand is one pair plus four melds (a triplet of identical tiles or a run of three consecutive ranks in one suit). Try every tile as the pair, then repeatedly take the smallest remaining tile: it can only be covered by a triplet of itself or by a run that starts at it, so there are at most two choices at each step and the search is tiny for 14 tiles.

Basis: Written for this question; the pair-plus-melds check is the same backtracking as LeetCode 846, Hand of Straights, plus a triplet option.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def is_win(tiles):
    for t in set(tiles):
        if tiles.count(t) >= 2:
            rest = remove(tiles, [t, t])
            if all_melds(rest):
                return True
    return False
def all_melds(tiles):
    if not tiles:
        return True
    first = min(tiles)
    for meld in (triple(first), run(first)):
        if contains(tiles, meld):
            if all_melds(remove(tiles, meld)):
                return True
    return False
```

TIME	SPACE
roughly C(12,3) * C(9,3) ... roughly C(12,3) * C(9,3) * C(6,3) per pair	**O(1)**

WHY IT IS SLOW

Tries every set of three tiles at every step. The smallest remaining tile can only be in a triplet of itself or a run starting at it, so at most two choices remain.

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

TIME	SPACE
O(1) for 14 tiles (about 2^4 branches per pair)	**O(distinct tiles)**

OPTIMAL SOLUTION · Try each pair, then greedy-smallest with two options

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A variant needs only two melds plus a pair (8 tiles of 14).

Ask what happens to the other six tiles before coding: if they are free, check whether any 8 tiles form a pair and two melds with the same search over subsets of the 14. The rule set is intentionally vague, so confirm melds, honours and flowers first.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import Counter


def is_run_start(tile, count):
    # numbered suits are 0..2 with ranks 1..9; the tile
    # id is suit * 10 + rank. Honours (30+) never run.
    ok = tile < 30 and tile % 10 <= 7
    nxt, nxt2 = count.get(tile + 1), count.get(tile + 2)
    return ok and nxt and nxt2


def melds_left(count, need):
    # can the tiles in `count` form exactly `need` melds?
    live = [t for t in sorted(count) if count[t]]
    tile = live[0] if live else None
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3
        if ok:
            return True
    # ... or starts a run of three
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1
        return ok
    return False


# BASE QUESTION (optimal): Try each pair, then greedy-
# smallest with two options
# FOLLOW-UP 1: A variant needs only two melds plus a pair
# (8 tiles of 14).
def is_winning_hand(hand, melds=4):
    # FOLLOW-UP 1: melds=2 handles the 8-tile variant
    # one pair plus `melds` melds uses every tile
    if len(hand) != 2 + 3 * melds:
        return False
    count = Counter(hand)
    # try every tile value as the pair
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False


if __name__ == "__main__":
    win = [1, 1, 1, 2, 3, 4, 11, 12, 13,
           21, 22, 23, 31, 31]
    assert is_winning_hand(win)
    # 2, 3, 5 is not a run
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13,
            21, 22, 23, 31, 31]
    assert not is_winning_hand(lose)
    # honours only make triplets or the pair
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33,
               34, 34, 34]
    assert is_winning_hand(honours)
    # the smaller variant: a pair and two melds (8 tiles)
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    # wrong tile count is never a win
    assert not is_winning_hand(win[:13])
```

NOTES FROM CANDIDATES

Clarify the rules — the prompt is intentionally sparse. Confirm: are honor tiles (winds/dragons) included? Do flowers count? Multi-suit?
Greedy + backtracking is the standard approach:

Try every possible pair (tile, tile); remove the pair.
On the remaining 12 tiles, repeatedly take the smallest remaining tile and try to form a triplet of it or a run starting at it; backtrack on failure.

For 14 tiles this runs in well under 1 ms; the interviewer cares more about the case analysis and clean recursion than micro-optimizations.
The same algorithm extends to special hands (seven pairs / ????) if the interviewer adds them as a follow-up.

PREPARATION

Practice writing the backtracking decomposer for tiles / counters; the canonical "expression add operators" recursion trains the same backtracking skeleton.
Have a tile-frequency-map mental model — sorting is the alternative but harder to reason about for runs across suits.
Pre-plan for the inevitable rules clarification: pull out at least 3 questions before coding.

MEDIUM | 1Point3Acres | onsite-coding | 18 of 33

Minimum Direction Violations in a Directed Graph

ASKED 2026-07-19 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 10.6 | 30 DAYS 0 | PRIORITY 164/224

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: written and tested here for this exact question

LeetCode match: LC 1466 Reorder Routes to Make All Paths Lead to the City Zero (related)

Frequency: Single report | Last asked: 2026-07-19 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/minimum-direction-violations-directed-graph

QUESTION

EXAMPLES

```
n = 5
edges = [[0, 1], [2, 1], [2, 3], [4, 3]]
start = 0
end = 4

path: 0 -> 1 -> 2 -> 3 -> 4
cost: 0 + 1 + 0 + 1 = 2
output: 2
```

APPROACH

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what a violation is (traversing an edge against its direction), whether edge costs are all 1, what to return when the end cannot be reached, and whether the graph has parallel edges or self loops.

BRUTE FORCE TO SAY OUT LOUD

Plain BFS ignoring direction minimizes edges, not violations, so say why it is wrong. Dijkstra with costs 0 and 1 works at O((n + m) log n).

OPTIMAL IDEA

Add every edge twice: forward with weight 0, backward with weight 1. Run 0-1 BFS with a deque: relax a weight-0 edge by pushing to the front, a weight-1 edge to the back. The deque always holds nodes whose distances differ by at most 1, so O(n + m).

DRY-RUN THESE CASES

INTERVIEW-DAY PLAN

Start equals end (0). A graph with no path. A path that needs every edge reversed. A shortcut that costs more edges but fewer violations.

LIKELY FOLLOW-UPS

General edge costs (Dijkstra), shortest path itself (parent pointers), or negative costs (Bellman-Ford).

PITFALLS

Using visited flags instead of a distance check. Pushing weight-0 edges to the back.

HOW TO THINK ABOUT IT

Going against an edge costs 1 and going with it costs 0, so it is a shortest-path problem with costs of only 0 and 1. Plain BFS counts edges, not violations, so it is the wrong tool. A deque lets 0-cost moves jump the queue (push front) and 1-cost moves wait their turn (push back), which is Dijkstra without the heap.

Basis: Related to LeetCode 1466, Reorder Routes: a shortest path where traveling an edge backwards costs 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(n, edges, start, end):
    arcs = []
    for u, v in edges:
        arcs.append((u, v, 0))  # along the edge
        arcs.append((v, u, 1))  # against it
    dist = [INF] * n
    dist[start] = 0
    for _ in range(n - 1):
        for u, v, w in arcs:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
    return dist[end]
```

TIME	SPACE
O(V * E)	O(V)

WHY IT IS SLOW

Every round scans all edges, V - 1 rounds: O(V * E). Costs of only 0 and 1 allow a deque search (0-1 BFS) in O(V + E).

OPTIMAL SOLUTION · 0-1 BFS on the doubled graph

TIME	SPACE
O(n + m)	O(n + m)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Why is 0-1 BFS correct, and how does it compare with Dijkstra?

The deque always holds nodes whose distances differ by at most one: front elements are at distance d, back elements at d + 1. Relaxing a zero-weight edge keeps the distance at d (front), a one-weight edge makes d + 1 (back). That is the same order a heap would produce, so it is Dijkstra without the log factor: O(n + m) versus O((n + m) log n).

FOLLOW-UP 2 What if edge costs can be arbitrary non-negative numbers?

Use Dijkstra with a heap on the same doubled graph (forward cost = 0 or the edge cost, backward cost = the violation cost). With negative costs use Bellman-Ford and watch for negative cycles.

FOLLOW-UP 3 Edge costs are arbitrary non-negative numbers (different for each direction).

Use Dijkstra with a heap on the same doubled graph, giving each arc its own cost. 0-1 BFS only works because the costs are 0 or 1.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import deque

INF = float("inf")


# BASE QUESTION (optimal): 0-1 BFS on the doubled graph
def min_violations(n, edges, start, end):
    # 0-1 BFS: following an edge is free, reversing costs
    # 1
    graph = [[] for _ in range(n)]
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    dist = [INF] * n
    dist[start] = 0
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # cost 0 goes to the front, cost 1 to the
                # back
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return -1 if dist[end] == INF else dist[end]


# FOLLOW-UP 3: Edge costs are arbitrary non-negative
# numbers (different for each ...
def cheapest_route(n, edges, start, end):
    # edges are (u, v, forward_cost, backward_cost)
    graph = [[] for _ in range(n)]
    for u, v, fwd, back in edges:
        graph[u].append((v, fwd))
        graph[v].append((u, back))
    dist = [INF] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
        d, u = heapq.heappop(heap)
        # skip outdated heap entries
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    return -1 if dist[end] == INF else dist[end]


if __name__ == "__main__":
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    # path 0 -> 1 -> 2 -> 3 -> 4 reverses two edges
    assert min_violations(5, edges, 0, 4) == 2
    assert min_violations(5, edges, 0, 0) == 0
    assert min_violations(3, [(0, 1)], 0, 2) == -1
    # the same graph with every reversal costing 1
    costed = [(u, v, 0, 1) for u, v in edges]
    assert cheapest_route(5, costed, 0, 4) == 2
    # different cost per direction
    assert cheapest_route(2, [(0, 1, 5, 2)], 0, 1) == 5
    assert cheapest_route(2, [(0, 1, 5, 2)], 1, 0) == 2
```

NOTES FROM CANDIDATES

Represent each original edge twice: the forward direction with weight 0 and the reverse direction with weight 1.
A regular BFS minimizes the number of edges, not the number of direction violations.
The linear-time target is O(n + m) time and space.

PREPARATION

Implement 0-1 BFS from memory and rehearse the deque invariant for zero-weight versus one-weight transitions.
Compare the proof and complexity with Dijkstra on the same transformed graph.
Test unreachable destinations, parallel edges, cycles, and start == end.

MEDIUM | IN APP LC 347 | 1Point3Acres | phone-screen, onsite-coding | 19 of 33

★ TOP 75: rank 29 in the Medium and Hard priority list

★ Top-K Frequent in Large Logs

ASKED 2026-07-19 | 83 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 58.8 | 30 DAYS 41.1 | PRIORITY 20/224

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #347)

LeetCode match: LC 347 Top K Frequent Elements

Frequency: Medium | Last asked: 2026-07-19 | Roles: SWE, MLE

Original: https://www.1point3acres.com/interview/problems/company/google/top-k-frequent-from-logs

QUESTION

EXAMPLES

```
Input:  chatLog = ["A says hi", "B says hi", "A says
    bye", "C says hi", "A says yo"], k=2
Counts: A=3, B=1, C=1
Output: [A, B]  // or [A, C] — ties allowed
```

APPROACH

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

HOW TO THINK ABOUT IT

Count the speakers first; a log line is just a way to read the user id. Then you only need the K largest counts: a heap of size K does that without sorting all users. For logs spread over many machines the local top K are not enough on their own, so say what extra information you send.

Basis: LeetCode 347, Top K Frequent Elements (here the key is the speaker in each chat line).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(chat_log, k):
    names = [l.split()[0] for l in chat_log]
    counts = Counter(names)
    def rank(name):
        return (-counts[name], name)
    return sorted(counts, key=rank)[:k]
```

TIME	SPACE
O(n + d log d)	O(d)

BRUTE FORCE

WHY IT IS SLOW

Sorts all d distinct speakers, O(d log d), when only K are wanted; a size-K heap is O(d log k), and a stream needs a counter sketch instead of exact counts.

OPTIMAL SOLUTION · Count, then keep a size-K heap

TIME	SPACE
O(n + d log k)	O(d)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you do it across many machines when the log does not fit on one?

Count per shard and send each shard's local top-K to a coordinator. That is not exact in general (a user can be 11th everywhere yet first overall), so for an exact answer use the threshold algorithm: gather the local top-K, compute a global lower bound, ask every shard for all users above bound / numShards, and merge. If an approximate answer is acceptable, merge count-min sketches (they add) and keep a heap of the K largest estimates.

FOLLOW-UP 2 What if the log arrives as an unbounded stream?

Use the Misra-Gries or Space-Saving summary with m counters. Any user with more than n / (m + 1) messages is guaranteed to be in the summary, and a second pass (or a count-min sketch for the estimate) can confirm the real counts. **Cost:** O(1) per message with O(m) memory for m counters (Misra-Gries), instead of O(d) memory for the exact counts.

FOLLOW-UP 3 Which approach would you choose and why?

A heap is the safe default: simple, O(n + d log k) and streaming-friendly. Quickselect wins when k is a large fraction of d and you need all K but not their order. Sorting is fine only when d is small. Say that out loud, with the complexities, because the interviewer is grading the trade-off discussion.

FOLLOW-UP 4 The log is a stream and memory is limited.

Misra-Gries keeps m counters; any user with more than n / (m + 1) messages is guaranteed to remain in the summary (checked above). A second pass, or a count-min sketch, then gives the real counts of the survivors.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq
from collections import Counter
```

```python
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def rank_key(item):
    # more messages first; ties by name
    return (-item[1], item[0])


# BASE QUESTION (optimal): Count, then keep a size-K heap
def top_k_speakers(chat_log, k):
    counts = Counter(speaker(line) for line in chat_log)
    # a heap of size k: O(n + d log k)
    items = counts.items()
    best = heapq.nsmallest(k, items, key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 1: How do you do it across many machines when
# the log does not fit on ...
def merge_shards(shard_logs, k):
    # exact answer: add the full counts of every shard
    total = Counter()
    for log in shard_logs:
        total.update(speaker(line) for line in log)
    best = heapq.nsmallest(k, total.items(), key=rank_key)
    return [user for user, _ in best]


# FOLLOW-UP 2, 4: What if the log arrives as an unbounded
# stream?
class MisraGries:
    """m counters: a user with over n / (m + 1) of the
    n messages always stays in the summary."""

    def __init__(self, m):
        self.m, self.counts = m, {}

    def add(self, user):
        if user in self.counts:
            self.counts[user] += 1
        elif len(self.counts) < self.m:
            self.counts[user] = 1
        else:
            # no room: every counter pays one
            for u in list(self.counts):
                self.counts[u] -= 1
                if self.counts[u] == 0:
                    del self.counts[u]

    def candidates(self):
        return set(self.counts)


if __name__ == "__main__":
    log = ["A says hi", "B says hi", "A says bye",
           "C says hi", "A says yo"]
    assert top_k_speakers(log, 2) == ["A", "B"]
    assert top_k_speakers(log, 5) == ["A", "B", "C"]
    assert top_k_speakers([], 3) == []
    # two shards that each rank someone else first
    s1 = ["A says x", "B says x"]
    s2 = ["B says y", "B says z"]
    assert merge_shards([s1, s2], 1) == ["B"]
    mg = MisraGries(2)
    for u in "AAABBC":
        mg.add(u)
    # A has 3 of 6 messages (> 6 / 3), so it must survive
    assert "A" in mg.candidates()
```

NOTES FROM CANDIDATES

The interviewer typically wants a discussion of the three implementations + complexity trade-off, not just one working solution.
For the distributed follow-up, candidates that mention count-min sketch and topK per shard -> global merge got positive feedback.
The same family surfaces as "Top-K IPs", "Top-K chatters", and "Top-K words"; the technique is interchangeable.
A recent variant uses a gigabyte-scale IP list and explicitly requires exact time and space complexity analysis.

PREPARATION

Practice writing quickselect with the partition step (and explain how to recover from a bad pivot).
Be able to state amortized vs worst-case time for each approach.
Have a one-line answer for streaming: "hashmap counts + heap of size K maintained online".
Drill the distributed answer once: per-shard top-K, then K-way merge of the partial heaps.

MEDIUM | 1Point3Acres | phone-screen | 20 of 33

Count Word Decompositions into Chemical Element Symbols

ASKED 2026-07-14 | 88 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 43.5 | 30 DAYS 0 | PRIORITY 80/224

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: written and tested here for this exact question

Related in your app (a neighbouring problem, not this exact question): LeetCode #139 Word Break

LeetCode match: LC 139 Word Break (related)

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/periodic-table-word-decomposition

QUESTION

EXAMPLES

Physics has four stated decompositions

P - H - Y - Si - C - S P - H - Y - Si - Cs P - H - Y - S - I - C - S P - H - Y - S - I - Cs The returned count is 4.

APPROACH

Implement a suffix-count or prefix-count formulation and state what each state represents.
Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether matching is case-insensitive, whether to count decompositions or just test for one, and for the dictionary (the periodic table symbols of one and two letters).

BRUTE FORCE TO SAY OUT LOUD

Recursive search that tries both a one-letter and a two-letter symbol at each position. Exponential in the worst case.

OPTIMAL IDEA

Suffix-count DP: ways[i] is the number of decompositions of word[i:], with ways[n] = 1. ways[i] adds ways[i + 1] if word[i] is a symbol and ways[i + 2] if word[i:i + 2] is a symbol. O(n). Physics gives 4.

DRY-RUN THESE CASES

Physics (4), a word with no decomposition (hello), one with exactly one, the empty string, overlapping one- and two-letter symbols such as Si versus S + I.

LIKELY FOLLOW-UPS

List the decompositions, handle a larger dictionary with a trie, or count modulo a prime.

PITFALLS

Forgetting to lowercase before the lookup. Starting the DP with ways[n] = 0.

HOW TO THINK ABOUT IT

Counting the ways to cut a string into dictionary pieces is Word Break with a count instead of a yes/no. Define ways[i] as the number of decompositions of the suffix starting at i; it depends only on ways[i + 1] and ways[i + 2], because symbols have one or two letters. Compute from the end so every value you need already exists.

Basis: A counting version of LeetCode 139, Word Break, with the periodic-table symbols as the dictionary.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def ways(i):
    if i == len(word):
        return 1
    total = 0
    if word[i] in SYMBOLS:
        total += ways(i + 1)
    if word[i:i + 2] in SYMBOLS:
        total += ways(i + 2)
    return total
```

TIME	SPACE
exponential (Fibonacci gr... exponential (Fibonacci growth)	**O(n)**

WHY IT IS SLOW

The same suffix is reached by many different splits, so the calls grow like the Fibonacci numbers. Computing ways from the end of the word once per position is O(n).

OPTIMAL SOLUTION · Suffix-count DP

TIME	SPACE
O(n)	**O(n)**

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How would you list the decompositions, not just count them?

Keep the same suffix recursion but return the list of splits instead of a number, memoized by start index. The count version is O(n); the list version is output-sensitive because there can be exponentially many splits, so agree on whether the interviewer wants a count, one witness, or all of them.

FOLLOW-UP 2 What if the dictionary is large and symbols can be long?

Put the dictionary in a trie (or a set plus the maximum word length L) and only look at substrings up to length L from each index. That makes the DP O(n * L) instead of O(n^2), which is the standard Word Break optimization.

FOLLOW-UP 3 List the decompositions instead of counting them.

Same recursion, but carry the parts chosen so far and record them at the end. The number of results can be exponential, so ask whether a count or a list is wanted.

FOLLOW-UP 4 A larger dictionary with long entries.

Store the dictionary in a set plus the maximum word length L (only look at substrings up to L), or in a trie, so each position costs O(L) lookups instead of O(n).

FOLLOW-UP 5 Count modulo a prime, or only test whether any decomposition exists.

Replace addition by addition modulo the prime, or by logical or for a yes/no answer. The DP structure is the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# BASE QUESTION (optimal): Suffix-count DP
def count_decompositions(word, symbols=ELEMENTS,
                         mod=None):
```

```
    w, n = word.lower(), len(word)
    # ways[i]: ways to split the suffix starting at i
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        # a one-letter symbol, then a two-letter symbol
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] %= mod
    return ways[0]


# FOLLOW-UP 5: Count modulo a prime, or only test whether
# any decomposition exists.
def exists_decomposition(word, symbols=ELEMENTS):
    # yes / no only: the same DP with or instead of +
    return count_decompositions(word, symbols) > 0


# FOLLOW-UP 3: List the decompositions instead of counting
# them.
def list_decompositions(word, symbols=ELEMENTS):
    w, n, memo = word.lower(), len(word), {}

    def splits(i):
        # every way to split the suffix starting at i
        if i == n:
            return [[]]
        if i in memo:
            return memo[i]
```

```
        out = []
        for size in (1, 2):
            part = w[i:i + size]
            if len(part) == size and part in symbols:
                for rest in splits(i + size):
                    out.append([part] + rest)
        memo[i] = out
        return out

    return splits(0)


# FOLLOW-UP 4: A larger dictionary with long entries.
def count_with_max_len(word, symbols):
    # big dictionary: only look at pieces up to its
    # longest
    longest = max(map(len, symbols))
    w, n = word.lower(), len(word)
    ways = [0] * (n + 1)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        for size in range(1, min(longest, n - i) + 1):
            if w[i:i + size] in symbols:
                ways[i] += ways[i + size]
    return ways[0]


if __name__ == "__main__":
    assert count_decompositions("Physics") == 4
    parts = list_decompositions("Physics")
    assert len(parts) == 4
    assert ["p", "h", "y", "si", "c", "s"] in parts
    assert exists_decomposition("Physics")
    # no element starts with x then q
    assert not exists_decomposition("xq")
    assert count_decompositions("Physics", mod=3) == 1
    assert count_with_max_len("Physics", ELEMENTS) == 4
```

NOTES FROM CANDIDATES

Confirm whether matching is case-sensitive and whether the caller supplies the symbol set or the implementation may treat the periodic table as fixed.
The result counts decompositions, not merely whether at least one decomposition exists.

PREPARATION

Implement a suffix-count or prefix-count formulation and state what each state represents.
Test a word with no decomposition, one decomposition, multiple decompositions, and overlapping one- and two-letter symbols.

MEDIUM | 1Point3Acres | onsite-coding | 21 of 33

Piano Hand Movement Grouping

ASKED 2026-07-14 | 88 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 201/224

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Low | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/piano-hand-movement-grouping

QUESTION

EXAMPLES

```
With notes = [1, 3, 5, 7, 9, 11] and k = 4, the
    stated output is 2.
Earlier examples use [1,2,3,4,5] -> 0 and [5,9,1] ->
    1, without stating the reach parameter.
```

APPROACH

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

INTERVIEW-DAY PLAN

CLARIFY FIRST

The statement is ambiguous, so ask: does the hand cover a fixed interval of keys [p, p + k], is the interval fixed until the hand lifts, does the first placement count as a move, and do notes arrive in the given order. Notice the examples are not all consistent, so ask which one is the reference.

BRUTE FORCE TO SAY OUT LOUD

Try every starting position for each group and take the minimum. Exponential or quadratic.

OPTIMAL IDEA

INTERVIEW-DAY PLAN

Greedy one-pass scan: start a group at the first note with the hand covering [note, note + k]. For each next note, if it is inside the interval keep going; if not, start a new group at that note. The answer is the number of groups (or groups minus one if the first placement is free). Group boundaries come from the same scan. O(n).

DRY-RUN THESE CASES

A single note, the example [1, 3, 5, 7, 9, 11] with k = 4, notes going down and then up, k = 0.

LIKELY FOLLOW-UPS

A hand that can slide, two hands, or return the group boundaries.

PITFALLS

Counting the initial placement inconsistently. Assuming notes are sorted.

HOW TO THINK ABOUT IT

Turn the physical rule into an invariant: the current group is valid while (largest note - smallest note) <= k, because you can place the hand at the smallest. The first note that breaks it must start a new group, and starting later never helps, so a single greedy pass is optimal. Ask whether the first placement counts as a move.

Basis: Written; interval grouping, like LeetCode 452 and 1326. Assumed rule (confirm it, the examples disagree): a hand placed on key p covers p to p + k inclusive and stays put until it is lifted; the notes come in order; the answer is the minimum number of placements, and the number of lifts is one fewer.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = [INF] * (n + 1)
best[0] = 0
for j in range(1, n + 1):
    for i in range(j):
        part = notes[i:j]
        if max(part) - min(part) <= k:
            best[j] = min(best[j], best[i] + 1)
answer = best[n]
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Every end tries every start and re-checks the range, O(n^2). A greedy pass that starts a new group only when the next note no longer fits is O(n).

OPTIMAL SOLUTION · Greedy scan, tracking the group's smallest and largest note

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where each group starts and where to place the hand.

Record the start index when a group opens and emit (start, end, smallest note) when it closes. The state transition is unchanged.

FOLLOW-UP 2 Count lifts, not placements.

Lifts are placements minus one if the first placement is free; the examples differ on this, so ask and state it. In code it is a single subtraction on the group count.

FOLLOW-UP 3 The hand may slide without a lift (costing something).

Then it becomes a DP over positions with a cost per slide distance, solved with the O(n * range) DP or by greedy if the cost is monotone. Ask for the cost model.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
def hand_positions(notes, k):
    # split the notes into groups that one hand position
    # (a span of at most k) can play, as early as possible
    groups = []
    for i, x in enumerate(notes):
        if groups:
            start, lo, hi = groups[-1]
            # does the note fit the current span?
            lo2, hi2 = min(lo, x), max(hi, x)
            if hi2 - lo2 <= k:
                groups[-1] = (start, lo2, hi2)
                continue
        # otherwise the hand must move: a new group starts
        groups.append((i, x, x))
    return groups


# BASE QUESTION (optimal): Greedy scan, tracking the
# group's smallest and largest note
def groups(notes, k):
    # how many hand placements are needed
    return len(hand_positions(notes, k))


# FOLLOW-UP 2: Count lifts, not placements.
def lifts(notes, k):
    # the first placement is free; every move is a lift
    return max(groups(notes, k) - 1, 0)


# FOLLOW-UP 1: Return where each group starts and where to
```

```
# place the hand.
def plan(notes, k):
    # where each group starts and ends, and where to put
    # the hand (on the lowest note of the group)
    out, spots = [], hand_positions(notes, k)
    for n, (start, lo, hi) in enumerate(spots):
        end = spots[n + 1][0] - 1 if n + 1 < len(spots) \
            else len(notes) - 1
        out.append((start, end, lo))
    return out


if __name__ == "__main__":
    # the stated example: 2 placements for reach 4
```

```
    assert groups([1, 3, 5, 7, 9, 11], 4) == 2
    # the earlier examples, counted as lifts
    assert lifts([1, 2, 3, 4, 5], 4) == 0
    assert lifts([5, 9, 1], 4) == 1
    assert groups([], 4) == 0
    assert plan([1, 3, 5, 7, 9, 11], 4) == [(0, 2, 1),
                                            (3, 5, 7)]
    # a reach of 0 means a new placement for every change
    assert groups([2, 2, 3], 0) == 2
```

NOTES FROM CANDIDATES

Clarify whether a placement is fixed until the hand lifts, whether the covered interval may slide without counting as a lift, and whether the initial placement counts. These details materially change the answer.
The six-note example's explanation says a placement at 7 covers keys 7–11 but still counts another lift for 9 and 11. Treat that as an ambiguity to resolve before coding rather than silently assuming the output convention.
Maintain the feasible placement interval for the current group.
The follow-up printout should reuse the same grouping state instead of duplicating the movement logic.

PREPARATION

Practice turning the physical reach rule into an explicit interval invariant before writing code.
Implement a one-pass grouping scan, then extend it to emit group boundaries without changing the core state transition.

MEDIUM | 1Point3Acres | tech-screen | 22 of 33

Snake Case to Camel Case with Repository-Wide Refactor

ASKED 2026-07-14 | 88 DAYS AGO | LAST 90 DAYS

STUDY: READ | NO LEETCODE FREQUENCY | PRIORITY 209/224

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: written and tested here for this exact question

Frequency: Single report | Last asked: 2026-07-14 | Roles: SWE

Original: https://www.1point3acres.com/interview/problems/company/google/snake-case-repository-refactor

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

APPROACH

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

HOW TO THINK ABOUT IT

Converting foo_bar to fooBar is easy. The risk is in the migration: a plain text replace also changes strings, comments, third-party names and public interfaces. Work on tokens (or the syntax tree) instead, rename only symbols defined in your own code, keep an explicit allow-list of public names, and prove nothing broke by running the tests before and after.

Basis: Written for this question; the conversion is a one-liner and the real work is a safe, token-based migration (not plain text replace).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
for path in source_files(repo):
    text = read(path)
    for word in find_snake_words(text):
        if not is_third_party(word):
            new = to_camel(word)
            text = text.replace(word, new)
    write(path, text)
```

TIME	SPACE
O(size of the repository)	O(1)

WHY IT IS SLOW

Also rewrites strings, comments, third-party calls and public interface names. Renaming only the identifiers the code defines, found with the syntax tree and tokens, keeps those intact.

OPTIMAL SOLUTION · Convert names, rename only defined identifiers via tokens

TIME	SPACE
O(size of the source)	O(names)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 How do you prove the migration is safe?

Run the full test suite before and after (green both times), compare the public API surface (exported names) before and after, and migrate in small reviewable commits with a codemod that only touches owned symbols. Anything reachable by reflection, serialisation, config files or other repositories needs an explicit alias or a deprecation period rather than a silent rename.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import ast
import io
import tokenize


# BASE QUESTION (optimal): Convert names, rename only
# defined identifiers via tokens
def snake_to_camel(name):
    # foo_bar -> fooBar (input is a valid snake_case name)
    head, *rest = name.split("_")
    caps = [p[:1].upper() + p[1:] for p in rest]
    return head + "".join(caps)


# FOLLOW-UP 1: How do you prove the migration is safe?
def owned_names(source, keep=()):
    # names this code defines itself: safe to rename
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
            args = node.args.args + node.args.kwonlyargs
            names.update(a.arg for a in args)
        elif isinstance(node, ast.Name):
            if isinstance(node.ctx, ast.Store):
                names.add(node.id)
    # only real snake_case, never dunder or private names
    return {n for n in names if "_" in n.strip("_")
            and not n.startswith("_") and n not in keep}


# FOLLOW-UP 1: How do you prove the migration is safe?
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    out, prev = [], None
    # work on tokens: strings and comments are untouched
    for tok in tokenize.generate_tokens(
            io.StringIO(source).readline):
        # skip attribute names (after a dot): they may be
        # third-party, so they stay as written
        dot = prev is not None and prev.string == "."
        is_name = tok.type == tokenize.NAME
        if is_name and tok.string in owned and not dot:
            new = snake_to_camel(tok.string)
            tok = tok._replace(string=new)
        out.append(tok)
        prev = tok
    return tokenize.untokenize(out)


if __name__ == "__main__":
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    src = (
        "import os\n"
        "def load_data(file_path):\n"
        "    full_path = os.path.join(file_path, 'x_y')\n"
        "    return full_path  # load_data is fine\n"
        "result_value = load_data('a')\n")
    new = rename_identifiers(src, keep={"file_path"})
    assert "def loadData(file_path)" in new
    assert "fullPath = os.path.join" in new
    # strings, comments and third-party calls are intact
    assert "'x_y'" in new and "# load_data is fine" in new
    assert "resultValue = loadData('a')" in new
    # the migrated code still runs
    scope = {}
    exec(compile(new, "x", "exec"), scope)
    assert scope["resultValue"] == "a/x_y"
```

NOTES FROM CANDIDATES

The implementation is intentionally short. Most of the round is spent on scoping a safe refactor, validation strategy, symbol ownership, and compatibility boundaries.
Clarify whether names appear only as parsed identifiers or may also occur in strings, generated code, configuration, reflection, and serialized data.
A text replacement is not enough to distinguish local symbols from external APIs or public contracts.

PREPARATION

Implement the identifier conversion and enumerate edge cases before expanding scope.
Rehearse a migration plan covering symbol-aware rewriting, build and test validation, staged rollout, compatibility shims, and rollback.

PRACHUB

7 questions | Hard 2, Medium 4, Easy 1 | posted in the last 30 days: 0, last 90 days: 7 (grouped by recency, newest first) | topics: BFS/DFS and trees first

POSTED: 31 TO 90 DAYS AGO (7)

Hard (2)

HARD | IN APP LC 41 | PracHub | Technical Screen | 23 of 33

Smallest Missing Positive Integer in Linear Time and Constant Extra Space

POSTED 2026-09-09 | 31 DAYS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 48 30 DAYS 0 PRIORITY 72/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #41 First Missing Positive

LeetCode match: LC 41 First Missing Positive

Posted on PracHub: 2026-09-09 (updated 2026-09-28)

Original: https://prachub.com/coding-questions/smallest-missing-positive-integer-in-linear-time-and-constant-extra-space

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **O(n)** time and **O(1)** auxiliary space, where **n = len(nums)**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **1 <= len(nums) <= 10^5**
- **-2147483648 <= nums[i] <= 2147483647**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **nums = [4, -2, 1, 2]**
- Output: **3**
- Explanation: **1** and **2** are present and **3** is not.

Example 2

- Input: **nums = [5, 6, 100, -3]**
- Output: **1**
- Explanation: **1** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **nums = [1, 2, 2, 3]**
- Output: **4**
- Explanation: **1**, **2** and **3** all appear (**2** twice), so the smallest missing positive integer is **4**.

APPROACH

Cyclic placement: swap each value v in 1..n into index v-1 until settled, then the first index i where nums[i] != i+1 gives the answer (else n+1). O(n) time, O(1) space.

HOW TO THINK ABOUT IT

The answer is between 1 and n + 1, because n numbers can cover at most n of the positive integers. So the array itself can be used as the 'seen' table: put each value v (1 <= v <= n) at index v - 1. After that, the first index i where nums[i] != i + 1 gives the answer i + 1.

Basis: LeetCode 41, First Missing Positive.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums):
    seen = set(nums)
    x = 1
    while x in seen:
        x += 1
    return x
```

TIME	SPACE
O(n)	O(n)

WHY IT IS SLOW

O(n) time but O(n) extra memory, which is what the follow-up asks to remove. Placing each value v at index v - 1 inside the array itself, then finding the first mismatch, uses O(1) space.

OPTIMAL SOLUTION · Cyclic placement: put each value v at index v - 1

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Do it without swapping (sign marking).

Replace out-of-range values by n + 1, then for every value v in 1..n make the entry at index v - 1 negative as a 'seen' flag; the first positive entry marks the missing number. Same O(n) time and O(1) space.

FOLLOW-UP 2 The array must not be modified.

Then O(1) space with O(n) time is not possible in general; copy the array (O(n) space) or accept O(n log n) time with a sort of a copy. State the trade-off.

FOLLOW-UP 3 Smallest missing non-negative integer, or the k-th missing positive.

Shift the target range by one for non-negative. For the k-th missing positive in a sorted array use binary search on 'how many numbers are missing before index i' = a[i] - (i + 1).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from bisect import bisect_left


# BASE QUESTION (optimal): Cyclic placement: put each
# value v at index v - 1
def first_missing_positive(nums):
    # put each value v in 1..n at index v - 1 by swapping
    a, n = list(nums), len(nums)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            j = a[i] - 1
            a[i], a[j] = a[j], a[i]
    # the first index that does not hold index + 1
    for i in range(n):
        if a[i] != i + 1:
            return i + 1
    return n + 1


# FOLLOW-UP 1: Do it without swapping (sign marking).
def first_missing_marking(nums):
    # no swaps: use the sign of a[v - 1] as a "seen" flag
    a, n = list(nums), len(nums)
    # values outside 1..n cannot matter: replace them
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    # the first positive entry marks the missing number
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1


# EASY VERSION: O(n) memory (say it first)
def first_missing_set(nums):
    # the easy O(n) memory answer (fine as a first
    # version)
    seen, v = set(nums), 1
    while v in seen:
        v += 1
    return v


# VARIANT: k-th missing positive
def kth_missing_positive(sorted_nums, k):
    # follow-up: k-th missing positive in a sorted array.
    # missing before index i is a[i] - (i + 1): binary
    # search
    lo, hi = 0, len(sorted_nums)
    while lo < hi:
        mid = (lo + hi) // 2
        if sorted_nums[mid] - (mid + 1) < k:
            lo = mid + 1
        else:
            hi = mid
    return lo + k


if __name__ == "__main__":
    cases = [([3, 4, -1, 1], 2), ([1, 2, 0], 3),
             ([7, 8, 9, 11, 12], 1), ([], 1),
             ([1, 1, 1], 2), ([2, 1], 3)]
    for nums, want in cases:
        assert first_missing_positive(nums) == want
        assert first_missing_marking(nums) == want
        assert first_missing_set(nums) == want
    # the input list is left untouched
    original = [3, 4, -1, 1]
    first_missing_positive(original)
    assert original == [3, 4, -1, 1]
    # 2, 3, 4, 7, 11: the 5th missing positive is 9
    assert kth_missing_positive([2, 3, 4, 7, 11], 5) == 9
    assert kth_missing_positive([1, 2, 3, 4], 2) == 6
```

FUNCTION AND TEST CASES

Signature

```
def first_missing_positive(nums: list[int]) -> int:
```

Sample call

```
first_missing_positive([4, -2, 1, 2])
```

Test cases

Input	Expected
([4, -2, 1, 2],)	3
([5, 6, 100, -3],)	1

HARD | PracHub | Technical Screen | 24 of 33

Find the Longest Straight Run of One Color

POSTED 2026-08-23 | 48 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 19.6 30 DAYS 0 PRIORITY 144/224

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: the interview site's own solution

LeetCode match: LC 562 Longest Line of Consecutive One in Matrix

Posted on PracHub: 2026-08-23 (updated 2026-09-10)

Original: https://prachub.com/coding-questions/find-the-longest-straight-run-of-one-color

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **R** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

The down-left diagonal from the top-right cell contains three **G** cells. No horizontal or vertical sequence has that length.

APPROACH

Per-cell DP over four directions (horizontal, vertical, diagonal, anti-diagonal): run[d][i][j] = run[d][prev]+1 if same color else 1. O(m*n).

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether diagonals count in both directions (yes, anti-diagonals too), whether cells must share an exact color value, and what to return for an empty grid. Confirm that a run cannot bend or skip a cell.

BRUTE FORCE TO SAY OUT LOUD

For every cell and each of the four directions, walk forward while the color matches. That is O(m * n * min(m, n)) and is a fine first answer to say out loud.

OPTIMAL IDEA

Let run[d][i][j] be the length of the same-color run that ends at (i, j) going in direction d. It is run[d][prev] + 1 if the previous cell in that direction has the same color, else 1. Four directions need only the previous row, so memory can be O(n). Answer is the maximum over everything. Time O(m * n).

DRY-RUN THESE CASES

A single cell (answer 1). A 1 x n row of one color. A grid where the longest run is on the anti-diagonal. A grid where every neighbor differs (answer 1).

LIKELY FOLLOW-UPS

Return the cells of the run, not just the length. Allow one wildcard cell. Make the run length at least K and count how many such runs exist.

PITFALLS

Forgetting the anti-diagonal needs the up-right previous cell, so it must be filled in the right scan order. Counting a bent path as a run.

HOW TO THINK ABOUT IT

A run in a fixed direction depends only on the cell before it in that direction, so each of the four directions is a one-line recurrence (like longest common suffix). Only four directions are needed because a line read backwards is the same line. Rolling only the previous row keeps memory at O(n).

Basis: Written; the same recurrence as LeetCode 562, Longest Line of Consecutive One in Matrix, with any color instead of 1.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
best = 1
for r, c in all_cells(grid):
    for dr, dc in DIRECTIONS:  # 4 directions
        n = 1
        while same(grid, r, c,
                   r + n * dr, c + n * dc):
            n += 1
        best = max(best, n)
```

BRUTE FORCE

TIME	SPACE
O(m * n * min(m, n))	O(1)

WHY IT IS SLOW

A long run is re-walked from every cell inside it, O(m * n * min(m, n)). A table of the run length ending at each cell per direction is O(m * n).

OPTIMAL SOLUTION · DP per direction with a rolling row

TIME	SPACE
O(m * n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Return where the run is, not only its length.

Only start a walk at cells that begin a run (the previous cell in that direction has a different color); that keeps the total work linear, and you record the start cell and direction of the best run. Equivalent: keep the DP table and read the end cell of the maximum, then step back by its length.

FOLLOW-UP 2 Require the run to be at least K and count how many such runs exist.

Count a run once, at its start, when its length is at least K (use the begins-a-run test above). Counting at every cell would count a long run many times.

FOLLOW-UP 3 The grid is huge and arrives row by row.

The rolling-row version already needs only the previous row, so it works on a stream of rows with O(n) memory.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))


def runs(grid):
    # yield (length, start, direction) for every maximal
    # run;
    # a run is walked only from its first cell, so the
    # total
    # work is linear in the number of cells
    rows, cols = len(grid), len(grid[0])
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                pr, pc = r - dr, c - dc
                before = 0 <= pr < rows and 0 <= pc < cols
                if before and grid[pr][pc] == grid[r][c]:
                    # not the first cell of its run
                    continue
                n, a, b = 0, r, c
                while 0 <= a < rows and 0 <= b < cols \
                        and grid[a][b] == grid[r][c]:
                    n += 1
                    a, b = a + dr, b + dc
                yield n, (r, c), (dr, dc)


# BASE QUESTION (optimal): DP per direction with a rolling
# row
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    return max(n for n, _, _ in runs(grid))


# FOLLOW-UP 1: Return where the run is, not only its
# length.
def longest_run_where(grid):
    # the length, the start cell and the direction
    if not grid or not grid[0]:
        return 0, None, None
    return max(runs(grid), key=lambda x: x[0])


# FOLLOW-UP 2: Require the run to be at least K and count
# how many such runs exist.
def count_runs_at_least(grid, k):
    # each run is counted once, at its start
    if not grid or not grid[0]:
        return 0
    return sum(1 for n, _, _ in runs(grid) if n >= k)


# FOLLOW-UP 3: The grid is huge and arrives row by row.
def longest_run_rows(row_stream):
    # rows arrive one by one: keep only the previous row's
    # runs
    best, prev = 0, None
    h = v = d = a = None
    for row in row_stream:
        n = len(row)
        nh, nv = [1] * n, [1] * n
        nd, na = [1] * n, [1] * n
        for c in range(n):
            if c and row[c] == row[c - 1]:
                nh[c] = nh[c - 1] + 1
            if prev is not None:
                if row[c] == prev[c]:
                    nv[c] = v[c] + 1
                if c and row[c] == prev[c - 1]:
                    nd[c] = d[c - 1] + 1
                if c + 1 < n and row[c] == prev[c + 1]:
                    na[c] = a[c + 1] + 1
            best = max(best, nh[c], nv[c], nd[c], na[c])
        prev, h, v, d, a = row, nh, nv, nd, na
    return best


if __name__ == "__main__":
    g1 = ["RRRG", "RRBB", "GRRR", "BGBR"]
    assert longest_run(g1) == 4
    assert longest_run(["RBG", "BGR", "GRB"]) == 3
    assert longest_run(["R"]) == 1
    assert longest_run([]) == 0
    length, start, direction = longest_run_where(g1)
    assert length == 4
    # runs of length at least 3 are counted once each
    assert count_runs_at_least(g1, 3) == 4
    assert longest_run_rows(iter(g1)) == 4
```

FUNCTION AND TEST CASES

Signature

```
def longestColorRun(grid: list[str]) -> int:
```

Sample call

```
longestColorRun(['RRRG', 'RRBB', 'GRRR', 'BGBR'])
```

Test cases

Input	Expected
`(['RRRG', 'RRBB', 'GRRR', 'BGBR'],)`	4
`(['RBG', 'BGR', 'GRB'],)`	3

ALSO REPORTED · 1 more

Longest Same-Color Grid Sequence | 1Point3Acres | phone-screen | asked 2026-09-08 | Single report

https://www.1point3acres.com/interview/problems/company/google/longest-same-color-grid-line

Track the length of a same-color run ending at each cell in four directions: horizontal, vertical, down-right diagonal, and down-left diagonal. For each direction, extend the predecessor's run by one only when its color equals the current cell; otherwise start at one.
Process rows from top to bottom and columns left to right. Horizontal state reads the current row's previous column; the other directions read the previous row. Take the maximum over all cells and directions. This checks straight consecutive lines, not connected regions or paths that turn.
Four constant-time transitions per cell give O(rows * columns) time. Keeping all states uses O(rows * columns) space; rolling previous/current rows reduces auxiliary space to O(columns). Clarify empty-grid behavior before coding.

Preparation: Implement the four-direction DP in 15 minutes and trace the given grid to identify its longest line.
Compare the DP with a direct straight-line scan on small grids; include one row, one column, alternating colors, and both diagonal orientations.

Medium (4)

MEDIUM | PracHub | Technical Screen | 25 of 33

Sliding-Window Mean After Removing the Largest K Values

POSTED 2026-09-01 | 39 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 29.8 30 DAYS 0 PRIORITY 125/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #480 Sliding Window Median

ABOUT THIS QUESTION

LeetCode match: LC 480 Sliding Window Median (order statistics in a window)

Posted on PracHub: 2026-09-01 (updated 2026-09-23)

Original: https://prachub.com/coding-questions/sliding-window-mean-after-removing-the-largest-k-values

QUESTION

For every contiguous window of a fixed size, remove the largest **k** values by multiplicity and compute the mean of the remaining values. Process sliding updates incrementally rather than sorting each complete window again. Implement **`trimmed_window_means(nums: int[], window: int, k: int) -> string[]`**. Return means in window order as reduced fractions **`numerator/denominator`**, using a positive denominator. This exact fraction encoding is an explicit practice representation that avoids cross-language floating-point rounding differences.

Constraints & Assumptions

- **`1 <= len(nums) <= 200000`**, **`1 <= window <= len(nums)`**, and **`0 <= k < window`**.
- Values range from -1,000,000 through 1,000,000. Use a sufficiently wide integer for window sums.
- Remove exactly **k** occurrences, not **k** distinct values. Equal values on the partition boundary may be represented in either group as long as the multiset counts are correct.
- For **`k == 0`**, retain the whole window. A window that would retain zero values is excluded by the input contract.
- Reduce each fraction by the greatest common divisor of the absolute numerator and denominator. Zero is **`0/1`**; an integer mean such as 5 is **`5/1`**.
- Aim for **`O(n log window)`** processing with **`O(window)`** maintained state, apart from the returned results.

Examples

```
nums = [1,9,2,8], window = 3, k = 1 result = ["3/2","5/1"]
```

```
nums = [-1,1,3], window = 2, k = 0 result = ["0/1","2/1"]
```

Explain how insertions, removals, duplicate values, and rebalancing preserve the retained sum. For the reported large-data/stream follow-up, describe which state must remain in memory and why a new window does not require a full sort.

```
Maintain a size and an ordering invariant Partition the window into the removed largest values and the retained values. Keep their sizes correct and ensure no retained value is larger than a removed value.
```

APPROACH

Keep the window in two structures (top-k multiset and the rest) with running sums, using two heaps with lazy deletion or a balanced BST; mean = rest_sum / (w - k). O(n log w).

HOW TO THINK ABOUT IT

The mean needs two sums: the whole window and the sum of its K largest values. Keep the window in a sorted container; the K largest are its last K elements. When one value enters or leaves, only one element crosses the boundary between 'kept' and 'removed', so the sum of the K largest can be updated in O(1) after an O(log) search.

Basis: LeetCode 480 (Sliding Window Median) style order statistics; the mean of the window after dropping exactly K occurrences of the largest values, returned as reduced fractions.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(nums, w, k):
    means = []
    for i in range(len(nums) - w + 1):
        window = sorted(nums[i:i + w])
        kept = window[:w - k]  # drop k largest
        total = sum(kept)
        means.append(Fraction(total, len(kept)))
    return means
```

TIME	SPACE
O(n * w log w)	O(w)

WHY IT IS SLOW

Each window is sorted from scratch, O(n * w log w). A sorted list updated by one insertion and one removal, with the sum of the top k adjusted at the boundary, avoids it.

OPTIMAL SOLUTION · Sorted window with an incrementally maintained top-K sum

TIME	SPACE
O(n * (log w + w) in memmove, O(n log w) with a balanced tree)	O(w)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the update O(log w) in the worst case.

A Python list insert shifts elements (fast in practice, O(w) in theory). A balanced tree or a Fenwick tree over compressed values gives O(log w) insert, delete and 'sum of the top K' via a descent.

FOLLOW-UP 2 Remove the smallest K instead, or both ends (trimmed mean).

Maintain the sum of the K smallest as well (the first K elements of the same sorted list); the mean of the middle values is (total - top - bottom) / (w - 2K).

FOLLOW-UP 3 Duplicates on the boundary.

Removing exactly K occurrences, not K distinct values, is what the positional approach does: elements equal to the boundary value are interchangeable, so which copy sits in the top K does not matter for the sum. **Cost:** unchanged: O(w) per update for list insertion (O(log w) with a balanced tree).

FOLLOW-UP 4 Trim the smallest K as well (a trimmed mean).

Maintain the sum of the K smallest alongside the K largest with the mirror-image boundary updates; the mean of the middle values is (total - top - bottom) / (w - 2K). Checked against sorting every window.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from bisect import bisect_left, bisect_right
from fractions import Fraction


# FOLLOW-UP 4: Trim the smallest K as well (a trimmed
# mean).
class Window:
    """A sorted window that keeps the sum of its k largest
    and k smallest values while values come and go."""

    def __init__(self, k):
        self.k, self.s = k, []
        self.total = self.top = self.bottom = 0

    def insert(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_right(s, x)
        s.insert(p, x)
        self.total += x
        # top-k region is the last k places (new list)
        if old_len < k:
            self.top += x
        elif p >= old_len + 1 - k:
            # x entered it: the value at old_len - k left
            # it
            self.top += x - s[old_len - k]
        # bottom-k region is the first k places
        if old_len < k:
            self.bottom += x
        elif p < k:
            self.bottom += x - s[k]

    def remove(self, x):
        k, s = self.k, self.s
        old_len = len(s)
        p = bisect_left(s, x)
        self.total -= x
        if old_len <= k:
```

```
            self.top -= x
            self.bottom -= x
        else:
            if p >= old_len - k:
                # the value just below the region joins it
                self.top += s[old_len - k - 1] - x
```

```
            if p < k:
                self.bottom += s[k] - x
        del s[p]


def as_text(fr):
    return f"{fr.numerator}/{fr.denominator}"


# BASE QUESTION (optimal): Sorted window with an
# incrementally maintained top-K sum
def trimmed_window_means(nums, window, k, both=False):
    # mean of each window after dropping the k largest
    # (and, with both=True, the k smallest as well)
    w, out = Window(k), []
    for i, x in enumerate(nums):
        w.insert(x)
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - (2 * k if both else k)
            drop = w.top + (w.bottom if both else 0)
            mean = Fraction(w.total - drop, kept)
            out.append(as_text(mean))
    return out


def brute(nums, window, k, both=False):
    # sort every window: the check used by the demo
    out = []
    for i in range(len(nums) - window + 1):
        part = sorted(nums[i:i + window])
        part = part[k:-k] if both and k else part
        part = part[:len(part) - k] if not both else part
        mean = Fraction(sum(part), len(part))
        out.append(as_text(mean))
    return out


if __name__ == "__main__":
    nums = [5, 1, 4, 2, 8, 3, 3, 9]
    fast = trimmed_window_means
    assert fast(nums, 3, 1) == brute(nums, 3, 1)
    assert fast(nums, 4, 2) == brute(nums, 4, 2)
    # a reduced fraction: (1 + 4) / 2 = 5/2
```

```
    assert fast([5, 1, 4], 3, 1) == ["5/2"]
    # duplicates at the boundary: remove exactly k copies
    dup = [3, 3, 3, 3]
    assert fast(dup, 3, 1) == ["3/1", "3/1"]
    # trimmed mean: drop k smallest and k largest
    both = fast(nums, 5, 1, True)
    assert both == brute(nums, 5, 1, True)
```

FUNCTION AND TEST CASES

Signature

```
def trimmed_window_means(nums: list[int], window: int, k:
int) -> list[str]:
```

Sample call

```
trimmed_window_means([1, 9, 2, 8], 3, 1)
```

Test cases

Input	Expected
`([1, 9, 2, 8], 3, 1)`	`['3/2', '5/1']`
`([-1, 1, 3], 2, 0)`	`['0/1', '2/1']`

ALSO REPORTED · 2 more

Sliding Window Average After Dropping the Largest Values | PracHub | Technical Screen

https://prachub.com/coding-questions/sliding-window-average-after-dropping-the-largest-values

Compute sliding-window average excluding top k | PracHub | Technical Screen

https://prachub.com/coding-questions/compute-sliding-window-average-excluding-top-k

---

MEDIUM | DESIGN-STYLE (skippable) | PracHub | Technical Screen | 26 of 33

Implement a Reusable Locker Allocation System

POSTED 2026-08-14 | 57 DAYS AGO | LAST 90 DAYS

STUDY: TALK ALL TIME 8.1 30 DAYS 0 PRIORITY 166/224

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: the interview site's own solution

LeetCode match: LC 1845 Seat Reservation Manager

Posted on PracHub: 2026-08-14 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.
- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

QUESTION

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"
), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can r
emove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker suppo
rts validation and pickup.
```

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.

Constraints

1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'],
    ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

APPROACH

Min-heap of released locker numbers plus a counter for the next never-used number; store pops the smallest, pickup pushes back. Track the last successful store separately. O(log n) per op.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask what store and pickup return, what happens when a customer stores twice, when no locker is free, and whether numbers are reused smallest first.

BRUTE FORCE TO SAY OUT LOUD

Scan lockers 1..n for the first free one on every store: O(n) per operation.

INTERVIEW-DAY PLAN

OPTIMAL IDEA

Keep a min-heap of released locker numbers and a counter next_new for lockers never used. store: if the customer already has one return -1; take the heap minimum if the heap is not empty, otherwise next_new if it is at most n, else return -1; remember customer to locker in a dict. pickup: free the customer's locker by pushing it on the heap. O(log n) per operation, and memory depends on used lockers, not n.

DRY-RUN THESE CASES

Pickup then store reuses the smallest freed number. Double store by one customer. Store when full. Pickup with no locker.

LIKELY FOLLOW-UPS

n up to a billion (still fine because of the counter), thread safety with a lock, or lockers with sizes.

PITFALLS

Initializing the heap with all n numbers. Not removing the customer record on pickup.

HOW TO THINK ABOUT IT

'Smallest free number' with numbers coming back means a min-heap of returned numbers. Numbers that were never handed out do not need to be stored: keep a counter for the next unused number and use the heap first. That keeps memory proportional to the number of lockers in use, not to n.

Basis: Written; the smallest-available-number pattern (LeetCode 1845 Seat Reservation Manager, 2336 Smallest Number in Infinite Set).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
owner = {}  # customer -> locker
def store(c):
    if c in owner:
        return -1
    taken = set(owner.values())
    for locker in range(1, n + 1):
        if locker not in taken:
            owner[c] = locker
            return locker
    return -1
def pickup(c):
    owner.pop(c, None)
```

TIME	SPACE
O(n) per operation	O(n)

WHY IT IS SLOW

Each store scans up to n lockers, O(n). A min-heap of freed numbers plus a counter for the next unused one makes each operation O(log n).

OPTIMAL SOLUTION · Min-heap of released numbers plus a next-unused counter

TIME	SPACE
O(log n) per operation	O(lockers released and not reused)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 There are 10^9 lockers.

The counter and the heap already make the structure independent of n: nothing is allocated up front, and memory depends only on how many lockers have been released. **Cost:** O(log R) per operation and O(R) memory, where R is the number of lockers released and not yet reused; independent of n.

FOLLOW-UP 2 Many threads call store and pickup.

Protect the heap, the counter and the customer map with one lock so a locker number is never handed out twice (or fine-grained: a lock around the (heap, counter) pair).

FOLLOW-UP 3 Lockers of different sizes.

Keep one heap and counter per size class, and let a customer choose the smallest class that fits. The structure per class is the same.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq


# BASE QUESTION (optimal): Min-heap of released numbers
# plus a next-unused counter
class Lockers:
    def __init__(self, n):
        self.n = n
        # the next never-used locker number
        self.next_new = 1
        # released lockers, smallest on top
        self.free = []
        # customer -> locker number
        self.of = {}
        # locker of the latest successful store
        self.last_store = -1

    def store(self, customer):
        if customer in self.of:
            return -1
        if self.free:
            # a released locker is always smaller than any
            # new one
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker

    def pickup(self, customer):
        locker = self.of.pop(customer, None)
        if locker is None:
            return -1
        # it becomes available again at once
        heapq.heappush(self.free, locker)
        return locker

    def last(self):
        # failed operations never change this
        return self.last_store


def process(n, operations):
    box, out = Lockers(n), []
    for op in operations:
        if op[0] == "store":
            out.append(box.store(op[1]))
        elif op[0] == "pickup":
            out.append(box.pickup(op[1]))
        else:
            out.append(box.last())
    return out


# FOLLOW-UP 2: Many threads call store and pickup.
class SafeLockers(Lockers):
    """Many threads: one lock around every operation."""

    def __init__(self, n):
        import threading
        super().__init__(n)
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return super().store(customer)

    def pickup(self, customer):
        with self.lock:
            return super().pickup(customer)


if __name__ == "__main__":
    ops = [["store", "a"], ["store", "b"],
           ["pickup", "a"], ["store", "c"], ["last"]]
    assert process(3, ops) == [1, 2, 1, 1, 1]
    # nothing stored yet, and a missing pickup
    empty = [["last"], ["pickup", "x"]]
    assert process(2, empty) == [-1, -1]
    # full, and a customer who already has a locker
    full = [["store", "a"], ["store", "a"],
            ["store", "b"], ["store", "c"]]
    assert process(2, full) == [1, -1, 2, -1]
    # failed operations leave last() alone
    more = full + [["last"]]
    assert process(2, more)[-1] == 2
    # a huge locker bank costs nothing up front
    one = [["store", "a"], ["last"]]
    assert process(10 ** 9, one) == [1, 1]
    s = SafeLockers(2)
    assert s.store("a") == 1 and s.pickup("a") == 1
```

FUNCTION AND TEST CASES

Signature

```
def process_locker_operations(n: int, operations: list[list[str]]) -> list[int]:
```

Sample call

```
process_locker_operations(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
```

Test cases

Input	Expected
`(3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])`	`[1, 2, 1, 1, 1]`
`(5, [])`	`[]`

ALSO REPORTED · 1 more

Implement a Reusable Locker Allocation System | PracHub | Technical Screen

https://prachub.com/coding-questions/implement-a-reusable-locker-allocation-system-2

MEDIUM | PracHub | Technical Screen | 27 of 33

Count Prefix-Matching Simple Paths in a Labeled Graph

POSTED 2026-08-09 | 62 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 42.5 | 30 DAYS 0 | PRIORITY 82/224

ABOUT THIS QUESTION

Topic: BFS/DFS Graph and Tree Traversal and Shortest Paths

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #79 Word Search

LeetCode match: No direct match; related LC 79 Word Search (inferred from overview)

Posted on PracHub: 2026-08-09 (updated 2026-08-28)

Original: https://prachub.com/coding-questions/count-prefix-matching-simple-paths-in-a-labeled-graph

QUESTION

Overview: Count distinct simple graph paths whose labels match every nonempty prefix of a query word. Paths may start anywhere but cannot revisit a vertex, so the task rewards subset-state search and careful counting over a small labeled graph.

APPROACH

DFS/backtracking from every start vertex, extending a path only while the labels match the next query character and the vertex is unvisited (visited bitmask); count paths for every prefix length.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask whether the graph is directed, whether a path is counted as a vertex sequence, whether paths of every prefix length are counted (the statement says every nonempty prefix), and the size (small, since the state is a visited subset).

BRUTE FORCE TO SAY OUT LOUD

Enumerate all simple paths and then test labels. Exponential and wasteful.

OPTIMAL IDEA

DFS with backtracking from every start vertex. Extend a path only if the next vertex is unvisited (a bitmask of visited vertices) and its label equals the next character of the query word. Every time the path length is L, count one for prefix length L. The pruning on labels keeps the search small. Time depends on the number of matching paths, bounded by n * d^(L-1).

DRY-RUN THESE CASES

A query of length 1 (count of vertices with that label), a graph with a cycle (simple paths may not revisit), a label that appears nowhere, repeated letters in the query.

LIKELY FOLLOW-UPS

Longer queries (memoize on (vertex, mask, position) only helps if counting, not listing), or undirected edges.

PITFALLS

Counting a path that revisits a vertex. Counting an undirected path twice (once in each direction) without agreeing on it.

HOW TO THINK ABOUT IT

A path is only worth extending while its labels still match the query, so prune on the label before going deeper. Distinctness is tracked with a visited set (a bitmask for small graphs). Count at every depth, since each prefix length has its own answer.

Basis: Written; backtracking on a small labelled graph. Assumed rules (confirm them): directed edges; a path is a sequence of distinct vertices; for each L from 1 to the length of the query, count the paths of L vertices whose labels spell the first L letters of the query.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
count = 0
def dfs(path):
    global count
    if len(path) <= len(query):
        if matches_prefix(path):
            count += 1
    for nxt in graph[path[-1]]:
        if nxt not in path:
            dfs(path + [nxt])
for v in vertices:
    dfs([v])
```

TIME	SPACE
exponential	O(path)

WHY IT IS SLOW

It explores paths whose first label already disagrees with the query. Extending a path only when the next label matches the next query letter prunes those branches.

OPTIMAL SOLUTION · DFS with label pruning and a visited bitmask

TIME	SPACE
O(number of matching paths * degree)	O(path)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The edges are undirected.

Add every edge in both directions, as above. A path is then counted once per direction unless you divide the counts for length >= 2 by 2 (a path and its reverse are different sequences only if the query is not a palindrome, so say how you count).

FOLLOW-UP 2 Count only the paths that spell the whole query.

Return counts[-1] instead of the list. For long queries on bigger graphs, memoise on (vertex, depth) if repeated vertices are allowed (walks, not simple paths); for simple paths the visited set prevents memoisation.

FOLLOW-UP 3 The graph has up to 100,000 vertices.

Simple paths are exponential, so ask whether walks are allowed. For walks a DP over (depth, vertex) counts them in O(L * E), the standard way to handle large graphs.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from collections import defaultdict


# BASE QUESTION (optimal): DFS with label pruning and a
# visited bitmask
def prefix_paths(labels, edges, query, undirected=False):
    # counts[d]: simple paths whose first d + 1 labels
    # spell
    # the first d + 1 letters of the query
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
        # FOLLOW-UP 1: undirected=True adds both
        # directions
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)

    def go(u, depth, mask):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            # a simple path never revisits a vertex
            if mask >> w & 1:
                continue
            if labels[w] == query[depth + 1]:
                go(w, depth + 1, mask | 1 << w)

    # a path may start at any vertex with the first letter
    for s in range(len(labels)):
        if query and labels[s] == query[0]:
            go(s, 0, 1 << s)
    return counts


# FOLLOW-UP 2: Count only the paths that spell the whole
# query.
def whole_query_paths(labels, edges, query,
                      undirected=False):
    # only the paths that spell the entire query
    both = undirected
    counts = prefix_paths(labels, edges, query, both)
    return counts[-1] if counts else 0


# VARIANT: walks may revisit vertices
def prefix_walks(labels, edges, query):
    # walks may revisit vertices: a DP over (depth,
    # vertex)
    # costs O(L * E) and works for very large graphs
    adj = defaultdict(list)
    for u, v in edges:
        adj[u].append(v)
    ways = {v: 1 for v in range(len(labels))
            if query and labels[v] == query[0]}
    counts = [sum(ways.values())] if query else []
    for ch in query[1:]:
        nxt = defaultdict(int)
        for u, n in ways.items():
            for w in adj[u]:
                if labels[w] == ch:
                    nxt[w] += n
        ways = nxt
        counts.append(sum(ways.values()))
    return counts


if __name__ == "__main__":
    labels = ["a", "b", "a", "b"]
    edges = [(0, 1), (1, 2), (2, 3), (3, 0)]
    # "aba": directed square a>b>a>b>...
    assert prefix_paths(labels, edges, "aba") == [2, 2, 2]
    assert whole_query_paths(labels, edges, "aba") == 2
    # undirected: every edge works both ways
    und = prefix_paths(labels, edges, "ab", True)
```

```
    assert und == [2, 4]
    # a walk may reuse vertices, a simple path may not
    walks = prefix_walks(labels, edges, "abab")
    assert walks == [2, 2, 2, 2]
    five = prefix_paths(labels, edges, "ababa")
    assert five == [2, 2, 2, 2, 0]
    assert prefix_paths(labels, edges, "") == []
```

FUNCTION AND TEST CASES

Signature

```
def count_prefix_paths(labels: str, edges: list[list[int]]
, word: str) -> list[int]:
```

Sample call

```
count_prefix_paths('roteoreote', [[0, 1], [0, 2], [0, 3],
[1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')
```

Test cases

Input	Expected
`('roteoreote', [[0, 1], [0, 2], [0, 3], [1, 4], [1, 5], [3, 6], [4, 7], [5, 8], [6, 9]], 'trees')`	`[2, 2, 1, 1, 0]`
`('a', [], 'aa')`	`[1, 0]`

MEDIUM | PracHub | Technical Screen | 28 of 33

★ TOP 75: rank 71 in the Medium and Hard priority list

★ Expand Recursive Key-Value Placeholders in a Template

POSTED 2026-08-04 | 67 DAYS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 0 | PRIORITY 45/224

ABOUT THIS QUESTION

Topic: Expression Parsing and Evaluation

Optimal answer from: the interview site's own solution

Related in your app (a neighbouring problem, not this exact question): LeetCode #394 Decode String

LeetCode match: No direct match; related LC 394 Decode String (inferred from overview)

Posted on PracHub: 2026-08-04 (updated 2026-08-16)

Original: https://prachub.com/coding-questions/expand-recursive-key-value-placeholders-in-a-template

QUESTION

Overview: Expand %KEY% placeholders in a template, recursively resolving placeholders inside mapped values. Use memoized dependency traversal to preserve literal text, avoid repeated work, and explain cycle, unknown-key, and escaping policies.

APPROACH

DFS expansion with memoization per key and a 'visiting' set to detect cycles; leave unknown keys per the spec; keep literal text unchanged.

INTERVIEW-DAY PLAN

CLARIFY FIRST

Ask for the placeholder syntax (%KEY%), what to do with an unknown key (leave it as is or raise), how to write a literal percent sign (escaping such as %%), and how to handle cycles (raise an error).

BRUTE FORCE TO SAY OUT LOUD

Repeatedly run a regex replace on the whole string until nothing changes. It loops forever on a cycle.

OPTIMAL IDEA

Resolve each key with memoized DFS: resolve(key) expands the value of that key by scanning for placeholders and recursively resolving them, with a visiting set to detect cycles and a cache so each key is expanded once. Expand the template by scanning once and replacing each placeholder with resolve(name), leaving literal text untouched. Linear in the sizes of text and values.

DRY-RUN THESE CASES

A value that refers to another key, a cycle (A refers to B, B to A), an unknown key, a template with no placeholders, two placeholders adjacent, an escaped percent.

LIKELY FOLLOW-UPS

Escaping, recursion depth limit (iterative expansion), or limiting the output size to prevent blow-up.

PITFALLS

Re-expanding a key every time. Treating an unknown key and a cycle the same way.

HOW TO THINK ABOUT IT

Expanding one key can require expanding others, so define expand(key) recursively and remember results (memoization) so a key used many times is expanded once. A 'currently expanding' set detects cycles: meeting a key that is already on the stack means a cycle. Decide the policy for unknown keys and for literal percent signs up front.

Basis: Written. Assumed rules (confirm them): placeholders look like %KEY%; the value of a key may itself contain placeholders; an unknown key is left as is; a cycle raises an error.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

BRUTE FORCE

```
def expand(text):
    for _ in range(LIMIT):
        new = text
        for key, value in values.items():
            tag = "%" + key + "%"
            new = new.replace(tag, value)
        if new == text:
            return text
        text = new
    raise Error("cycle")
```

TIME	SPACE
O(depth * length)	O(length)

WHY IT IS SLOW

It re-expands the same key every time it appears and needs an arbitrary cap to stop on cycles. A memoised recursive expansion with a visiting set expands each key once and detects cycles exactly.

OPTIMAL SOLUTION · Memoised recursive expansion with a visiting set

TIME	SPACE
O(total size of all values + template)	O(number of keys)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 A literal percent sign must be possible.

Choose an escape such as %% and handle it in the same scan: when the text has %%, output one % and skip both characters; otherwise try to match a placeholder. Test percent signs next to placeholders.

FOLLOW-UP 2 An unknown key should be an error, or replaced by an empty string.

Make the policy a parameter: raise KeyError, return the placeholder unchanged (as coded), or return ''. State your default.

FOLLOW-UP 3 Values are huge and expansion explodes.

The output can be exponentially larger than the input (a key referring to the previous key twice, repeatedly). Check the expanded length as you go and fail past a limit, or return a lazy expansion.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import re


PATTERN = re.compile(r"%%|%(\w+)%")


# BASE QUESTION (optimal): Memoised recursive expansion
# with a visiting set
# FOLLOW-UP 1: A literal percent sign must be possible.
def expand(template, mapping, unknown="keep",
           limit=10 ** 6):
    # unknown: "keep" the text, "empty" it, or "error"
    cache, visiting = {}, set()

    def expand_text(text):
        def one(m):
            key = m.group(1)
            # FOLLOW-UP 1: %% is a literal percent sign
            if key is None:
                # %% is a literal percent sign
                return "%"
            if key not in mapping:
                # FOLLOW-UP 2: unknown key: keep, empty or
                # error
                if unknown == "error":
                    raise KeyError(key)
                if unknown == "empty":
                    return ""
                return m.group(0)
            return expand_key(key)
        out = PATTERN.sub(one, text)
        # FOLLOW-UP 3: huge values: cap the expansion
        # stop explosive expansions early
        if len(out) > limit:
            raise ValueError("expansion too large")
        return out

    def expand_key(key):
        # each key is expanded once and then cached
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError(f"cycle at {key}")
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]

    return expand_text(template)


if __name__ == "__main__":
    m = {"USER": "admin", "HOME": "/%USER%/home"}
    text = "I am %USER% My home is %HOME%"
    want = "I am admin My home is /admin/home"
    assert expand(text, m) == want
    # unknown keys: kept, emptied, or an error
    assert expand("a %X% b", m) == "a %X% b"
    assert expand("a %X% b", m, unknown="empty") == "a  b"
    try:
        expand("%X%", m, unknown="error")
        raise AssertionError("missing key not reported")
    except KeyError:
        pass
    # a literal percent, even next to a placeholder
    assert expand("100%% of %USER%", m) == "100% of admin"
    # a cycle is reported
    try:
        expand("%A%", {"A": "%B%", "B": "%A%"})
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
    # a key that doubles the previous one explodes
    boom = {"K0": "ab"}
    for i in range(1, 30):
```

```
        boom[f"K{i}"] = f"%K{i - 1}%%K{i - 1}%"
    try:
        expand("%K29%", boom, limit=1000)
        raise AssertionError("explosion not stopped")
    except ValueError:
        pass
```

FUNCTION AND TEST CASES

Signature

```
def expand_template(values: dict[str, str], template: str)
 -> str:
```

Sample call

```
expand_template({}, 'plain text')
```

Test cases

Input	Expected
`({}, 'plain text')`	`'plain text'`
`({'USER': 'admin', 'HOME': '/%USER%/home'}, 'I am %USER% My home is %HOME%')`	`'I am admin My home is /admin/home'`

ALSO REPORTED · 1 more

Template Variable Expansion with Follow-Ups | 1Point3Acres | onsite-coding | asked 2026-08-14 | Single report

https://www.1point3acres.com/interview/problems/company/google/template-variable-expansion-dependencies

The interviewer spent most of the round probing the delimiter and escaping contract before allowing implementation. State a compact grammar and get explicit agreement; otherwise a nominally easy parser can lose the coding budget to unresolved edge cases.

Preparation: Implement one-pass placeholder scanning under a written delimiter/escape contract.
Resolve a dependency chain in the map before processing the template, then add cycle detection.
Prepare three malformed-input cases and explain whether each is rejected, preserved literally, or interpreted as an escape.

Easy (1)

EASY | IN APP LC 729 | PracHub | Technical Screen | 29 of 33

Check an Interval Against Existing Intervals for Conflicts

POSTED 2026-08-08 | 63 DAYS AGO | LAST 90 DAYS

STUDY: TALK | ALL TIME 26.5 | 30 DAYS 0 | PRIORITY 131/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: the interview site's own solution

Also in your app: LeetCode #729 My Calendar I

LeetCode match: LC 729 My Calendar I (inferred from overview)

Posted on PracHub: 2026-08-08 (updated 2026-08-29)

Original: https://prachub.com/coding-questions/check-an-interval-against-existing-intervals-for-conflicts

QUESTION

Overview: Check whether one half-open candidate interval conflicts with any interval in a large unsorted collection. Respect strict start and end semantics so ranges that only touch at an endpoint do not conflict.

APPROACH

Half-open intervals [s,e) conflict iff new.start < e and s < new.end (touching endpoints do not conflict). One-off check on an unsorted collection is a single O(n) scan; for repeated queries keep a sorted structure (TreeMap/balanced BST or sorted list + bisect) and check only the neighbors. O(log n) per query.

HOW TO THINK ABOUT IT

Two half-open intervals [s1, e1) and [s2, e2) overlap exactly when s1 < e2 and s2 < e1. If the booked intervals are kept sorted and disjoint, only the neighbours at the insertion point can conflict, so a binary search replaces the scan.

Basis: LeetCode 729, My Calendar I (booking succeeds only when the new half-open interval overlaps nothing), and 731 for double booking.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
booked = []
def book(s, e):
    for s2, e2 in booked:
        if s < e2 and s2 < e:
            return False  # overlap
    booked.append((s, e))
    return True
```

BRUTE FORCE

TIME	SPACE
O(n) per booking	O(n)

WHY IT IS SLOW

Each booking scans all earlier bookings, O(n). With the intervals sorted, binary search finds the only two neighbours that could conflict.

OPTIMAL SOLUTION · Sorted starts with bisect: check only two neighbours

TIME	SPACE
O(log n) search, O(n) list insert	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Allow double booking but never triple booking (LeetCode 731).

Keep a difference map (+1 at start, -1 at end) and sweep it in sorted order; if the running count ever exceeds 2, undo the change and reject. O(n log n) per booking, or use a segment tree for faster.

FOLLOW-UP 2 Report the maximum number of overlapping bookings after each booking (LeetCode 732).

The same difference map, taking the maximum running count in the sweep. A segment tree with lazy range increments gives O(log n) per booking.

FOLLOW-UP 3 Closed intervals instead of half-open.

Then touching endpoints conflict, so the tests become <= instead of <. Say it, and keep the choice in one comparison.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
from bisect import bisect_right
from collections import defaultdict


def conflicts(candidate, intervals):
    # half-open [start, end): touching ends do NOT
    # conflict
    s, e = candidate
    return any(a < e and s < b for a, b in intervals)


# BASE QUESTION (optimal): Sorted starts with bisect:
# check only two neighbours
class Calendar:
    """A sorted calendar: O(log n) per check."""
    def __init__(self):
        self.starts, self.ends = [], []

    def book(self, start, end):
        i = bisect_right(self.starts, start)
        # the booking before must end by `start`
        if i > 0 and self.ends[i - 1] > start:
            return False
        # the booking after must start at or after `end`
        if i < len(self.starts) and self.starts[i] < end:
            return False
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True


# FOLLOW-UP 1: Allow double booking but never triple
# booking (LeetCode 731).
class DoubleBooking:
    """LeetCode 731: two overlaps are fine, three are not.
    LeetCode 732: also report the largest overlap."""

    def __init__(self, limit=2):
        self.delta = defaultdict(int)
        self.limit = limit

    def peak(self):
        # sweep the +1 / -1 changes in time order
        run = best = 0
        for t in sorted(self.delta):
            run += self.delta[t]
            best = max(best, run)
        return best

    def book(self, start, end):
        self.delta[start] += 1
        self.delta[end] -= 1
        if self.peak() > self.limit:
            # undo the change and reject the booking
            self.delta[start] -= 1
            self.delta[end] += 1
            return False
        return True


if __name__ == "__main__":
    taken = [(10, 20), (30, 40)]
    assert conflicts((15, 25), taken)
    assert not conflicts((20, 30), taken)
    assert conflicts((19, 31), taken)
    # sorted calendar
    c = Calendar()
    assert c.book(10, 20) and c.book(20, 30)
    assert not c.book(15, 25) and c.book(5, 10)
    # double booking is allowed, triple booking is not
    d = DoubleBooking()
    assert d.book(10, 20) and d.book(15, 25)
    assert not d.book(17, 18)
    assert d.peak() == 2
    # limit 1 is the plain no-overlap calendar
    one = DoubleBooking(limit=1)
    assert one.book(1, 5) and not one.book(4, 6)
```

FUNCTION AND TEST CASES

Signature

```
def hasIntervalConflict(candidate: list[int], intervals: l
ist[list[int]]) -> bool:
```

Sample call

```
hasIntervalConflict([4,7], [[1,3],[7,9]])
```

Test cases

Input	Expected
`([4,7],[[1,3],[7,9]])`	`False`
`([4,7],[[1,5],[8,9]])`	`True`

EXPONENT

4 questions | Hard 1, Medium 3, Easy 0 | last asked (approx) in the last 30 days: 0, last 90 days: 4 (grouped by recency, newest first) | topics: BFS/DFS and trees first

ASKED (ROUGH): 31 TO 90 DAYS AGO (4)

Hard (1)

HARD | IN APP LC 23 | Exponent | 30 of 33

Merge k sorted linked lists.

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 49.4 | 30 DAYS 0 | PRIORITY 70/224

ABOUT THIS QUESTION

Topic: Other: Design, Heaps, Intervals and Misc

Optimal answer from: your app's interview approach (LeetCode #23)

LeetCode match: LC 23 Merge k Sorted Lists

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age) | +816 interviews

Original: https://www.aced.io/questions/1167/merge-k-sorted-linked-lists

QUESTION

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.Interview experiences6 shared‹›Revolut · 2 months ago"Its good, but I was bad and dont really remember much. But all rounds was good. HR was nice. Tech guy on tech interview was nice too."Software Engineer · Entry Level / L3FFloQast · 2 months ago"Screening and the system design with

director are virtual, while on-site loop in the office. The process is well organi

APPROACH

Min-heap of the k list heads (or divide and conquer pairwise merging). O(N log k).

HOW TO THINK ABOUT IT

At any moment the next output node is the smallest among the current heads of the k lists, which is a min-heap of size k. Each node is pushed and popped once, so the total cost is N log k. A second way is to merge lists in pairs, halving the number of lists each round, with the same bound.

Basis: LeetCode 23, Merge k Sorted Lists.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(lists):
    values = []
    for head in lists:
        while head:
            values.append(head.val)
            head = head.next
    values.sort()
    dummy = tail = ListNode(0)
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next
```

TIME	SPACE
O(N log N)	O(N)

WHY IT IS SLOW

O(N log N) time and O(N) extra memory, ignoring that each list is already sorted. A min-heap of the k current heads is O(N log k) with O(k) memory.

OPTIMAL SOLUTION · Min-heap of the current heads

TIME	SPACE
O(N log k)	O(k)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Merge the lists in pairs instead of using a heap.

Repeatedly merge neighbouring pairs with the two-list merge; there are log k rounds, each touching all N nodes: O(N log k) time and O(1) extra space (no heap).

FOLLOW-UP 2 The lists are too large for memory (files or streams).

The heap approach already works on streams: it holds one item per list. Read each input through an iterator and write the output as you go; this is the merge phase of an external sort (heapq.merge does it). **Cost:** O(N log k) time and O(k) memory; one sequential read per list.

FOLLOW-UP 3 Why not merge the lists one by one into a running result?

Merging list i into a result of size about i * (N / k) costs O(i * N / k), so the total is O(k * N), far worse than O(N log k).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import heapq


class ListNode:
    def __init__(self, val=0, next=None):
        self.val, self.next = val, next


def make(values):
    dummy = tail = ListNode()
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    out = []
    while node:
        out.append(node.val)
        node = node.next
    return out


# BASE QUESTION (optimal): Min-heap of the current heads
def merge_k_heap(lists):
    # a heap holds the current head of every list; the
    # node
    # index breaks ties so nodes are never compared
    heap = [(n.val, i, n)
            for i, n in enumerate(lists) if n]
    heapq.heapify(heap)
    dummy = tail = ListNode()
    while heap:
        _, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            nxt = node.next
            heapq.heappush(heap, (nxt.val, i, nxt))
    return dummy.next


# HELPER: merge two lists
def merge_two(a, b):
```

```
    dummy = tail = ListNode()
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next


# FOLLOW-UP 1: Merge the lists in pairs instead of using a
# heap.
def merge_k_pairs(lists):
    # no heap: merge neighbours in pairs, log k rounds,
    # O(1) space
    lists = [n for n in lists if n]
    if not lists:
        return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            if i + 1 < len(lists):
                pair = (lists[i], lists[i + 1])
                merged.append(merge_two(*pair))
            else:
                merged.append(lists[i])
        lists = merged
    return lists[0]


# FOLLOW-UP 2: The lists are too large for memory (files
# or streams).
def merge_streams(iterators):
    # lists too big for memory: stream them (heapq.merge
    # keeps
    # one item per input), the merge step of an external
    # sort
    return heapq.merge(*iterators)


if __name__ == "__main__":
    data = [[1, 4, 5], [1, 3, 4], [2, 6]]
    want = [1, 1, 2, 3, 4, 4, 5, 6]
    got = to_list(merge_k_heap([make(v) for v in data]))
```

```
    assert got == want
    got = to_list(merge_k_pairs([make(v) for v in data]))
    assert got == want
    assert merge_k_heap([]) is None
    assert merge_k_pairs([None]) is None
    assert to_list(merge_k_heap([None, make([1])])) == [1]
    got = list(merge_streams(iter(v) for v in data))
    assert got == want
    got = to_list(merge_two(make([1, 3]), make([2])))
    assert got == [1, 2, 3]
```

OFFICIAL PAGE TEXT

Merge K Sorted Linked Lists. You are given an array of k linked-lists lists, each linked-list is sorted in ascending order. Your task is to merge all the linked-lists into one sorted linked-list and return it. Examples: If lists = [[1,4,5],[1,3,4],[2,6]], where each array represents a linked-list, the merged and sorted linked-list would be [1,1,2,3,4,4,5,6]. The output for the merged list should be in array form for simplicity. If lists = [], indicating an array of no linked-lists, the output should be []. If lists = [[]], representing an array with a single empty linked-list, the output should be []. If lists = [[],[1]], where the first linked-list is empty and the second contains a single element, the output should be [1]. If lists = [[-1,5,11],[],[6,10]], the merged and sorted linked-list should be [-1,5,6,10,11]. Constraints: k == lists.length 0 <= k <= 10^4 0 <= lists[i].length <= 500 10^4 <= lists[i][j] <= 10^4 lists[i] is sorted in an increasing order. In this video, Simon, Spotify Software Engineer, answers an interview question about merging k sorted linked lists.No answers contributed by the community yet.

Medium (3)

MEDIUM | IN APP LC 56 | Exponent | 31 of 33

★ TOP 75: rank 11 in the Medium and Hard priority list

★ Merge Intervals

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE ALL TIME 65.2 30 DAYS 41.1 PRIORITY 10/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #56)

LeetCode match: LC 56 Merge Intervals

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age) | +1219 interviews

Original: https://www.aced.io/questions/1602/merge-intervals

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each

interval, compare it with the last interval in th

APPROACH

Sort by start and merge overlapping intervals in one pass. O(n log n).

HOW TO THINK ABOUT IT

After sorting by start, an interval can overlap only the interval just before it in the output, because everything earlier ends before that one starts. So one pass with a 'current merged interval' is enough: extend its end, or start a new one.

Basis: LeetCode 56, Merge Intervals (intervals sharing a point merge).

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def merge(intervals):
    intervals = list(intervals)
    merged = True
    while merged:
        merged = False
        for a, b in combinations(intervals, 2):
            if overlaps(a, b):
                intervals.remove(a)
                intervals.remove(b)
                intervals.append(union(a, b))
                merged = True
                break
    return sorted(intervals)
```

BRUTE FORCE

TIME	SPACE
O(n^2) or worse	O(n)

WHY IT IS SLOW

Each pass compares all pairs, O(n^2), and several passes may be needed. Sorting by start and sweeping once is O(n log n).

OPTIMAL SOLUTION · Sort by start and sweep

TIME	SPACE
O(n log n)	O(n)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Insert one interval into an already merged list (LeetCode 57).

Copy the intervals that end before it, merge the ones that overlap, copy the rest: O(n) with no sort.

FOLLOW-UP 2 Intervals of touching integers ([1, 2] and [3, 4]) should merge.

Change the overlap test to start <= last_end + 1. For real-valued ranges that would be wrong, so state which you assume.

FOLLOW-UP 3 Too many intervals for memory.

Sort externally (external merge sort) and stream the merge: the one-pass sweep needs only the current interval in memory. **Cost:** O(n log n) external sort with bounded memory runs, then an O(n) streaming merge with O(1) memory.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```python
import heapq


# BASE QUESTION (optimal): Sort by start and sweep
def merge_intervals(intervals, touching_ints=False):
    out = []
    # sort by start, then extend the last interval while
    # they overlap
    for s, e in sorted(intervals):
        gap = 1 if touching_ints else 0
        if out and s <= out[-1][1] + gap:
            out[-1][1] = max(out[-1][1], e)
        else:
            out.append([s, e])
    return out


# FOLLOW-UP 1: Insert one interval into an already merged
# list (LeetCode 57).
def insert_interval(merged, new):
    # LeetCode 57: already merged and sorted, so O(n) and
    # no sort
    out, i, n = [], 0, len(merged)
    # 1) everything that ends before the new one starts
    while i < n and merged[i][1] < new[0]:
        out.append(merged[i])
        i += 1
    # 2) everything that overlaps it is absorbed
    s, e = new
    while i < n and merged[i][0] <= e:
        s, e = min(s, merged[i][0]), max(e, merged[i][1])
        i += 1
    out.append([s, e])
    # 3) the rest
    return out + merged[i:]


# FOLLOW-UP 3: Too many intervals for memory.
def merge_stream(sorted_intervals):
    # intervals arrive sorted by start (for example after
    # an
    # external sort): only the current one is kept in
    # memory
    cur = None
    for s, e in sorted_intervals:
        if cur and s <= cur[1]:
            cur[1] = max(cur[1], e)
        else:
            if cur:
                yield tuple(cur)
            cur = [s, e]
    if cur:
        yield tuple(cur)


# FOLLOW-UP 3: Too many intervals for memory.
def merge_many_sorted(sources):
    # several already-sorted files: a heap merge, then the
    # sweep
    return list(merge_stream(heapq.merge(*sources)))


if __name__ == "__main__":
    spans = [[1, 3], [2, 6], [8, 10], [15, 18]]
    got = merge_intervals(spans)
    assert got == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []
    got = merge_intervals([[1, 2], [3, 4]])
    assert got == [[1, 2], [3, 4]]
    got = merge_intervals([[1, 2], [3, 4]], True)
    assert got == [[1, 4]]
    got = insert_interval([[1, 3], [6, 9]], [2, 5])
    assert got == [[1, 5], [6, 9]]
    assert insert_interval([], [5, 7]) == [[5, 7]]
    stream = list(merge_stream([(1, 3), (2, 6), (8, 10)]))
    assert stream == [(1, 6), (8, 10)]
    parts = [[(1, 3), (8, 9)], [(2, 4), (9, 12)]]
    assert merge_many_sorted(parts) == [(1, 4), (8, 12)]
```

OFFICIAL PAGE TEXT

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples

```
intervals = [[1,3], [2,6], [8,10], [15,18]]
output: [[1,6], [8,10], [15,18]]

intervals = [[1,4], [4,5]]
output: [[1,5]]

intervals = [[1,4], [2,3]]
output: [[1,4]]

intervals = [[1,2], [3,4], [5,6], [7,8]]
output: [[1,2], [3,4], [5,6], [7,8]]
```

The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in the result list. If the current interval overlaps with the last interval in the result (i.e., the start of the current interval is less than or equal to the end of the last interval), merge them by updating the end of the last interval in the result to the maximum end time of both intervals. If there is no overlap, add the current interval to the result list as a separate interval. Return the Result: After processing all intervals, the result list contains the merged intervals. Time Complexity: The dominant factor in the time complexity is the sorting step, which takes O(n log n), where n is the number of intervals. The merging process involves a single pass through the sorted intervals, which takes O(n) time. Therefore, the overall time complexity is O(n log n). Space Complexity: The space complexity is O(n), which accounts for the space required to store the result list. This is because, in the worst case, none of the intervals overlap, and the result list will contain all the original intervals.In this mock interview video, Lewin (Dropbox SWE), answers the interview question, "Given an array of intervals, merge the overlapping intervals and return an array of the non-overlapping intervals that cover all intervals in the input."No answers contributed by the community yet.

MEDIUM | IN APP LC 49 | Exponent | 32 of 33

★ TOP 75: rank 23 in the Medium and Hard priority list

★ Group anagrams

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.7 | 30 DAYS 51.4 | PRIORITY 14/224

ABOUT THIS QUESTION

Topic: Core Array, String, Hash Map, Sliding Window, and Binary Search Patterns

Optimal answer from: your app's interview approach (LeetCode #49)

LeetCode match: LC 49 Group Anagrams (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4676/group-anagrams

QUESTION

Group anagrams

APPROACH

Hash map keyed by sorted letters (or a 26-count tuple) to lists of words. O(n * k log k).

HOW TO THINK ABOUT IT

Anagrams are strings with the same letters in any order, so they have the same canonical form: the sorted letters, or the count of each letter. Use that as a dictionary key and group in one pass; you never compare two strings directly.

Basis: LeetCode 49, Group Anagrams.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def brute(words):
    groups = []
    for word in words:
        key = sorted(word)
        for group in groups:
            if sorted(group[0]) == key:
                group.append(word)
                break
        else:
            groups.append([word])
    return groups
```

TIME	SPACE
O(n * g * L log L)	O(n)

WHY IT IS SLOW

Each word is compared with many groups, O(n * groups * L log L). Using the sorted letters (or a letter-count tuple) as a dictionary key groups all words in one pass.

OPTIMAL SOLUTION · Dictionary keyed by the sorted letters

TIME	SPACE
O(n * L log L)	O(n * L)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 Make the key O(L) instead of O(L log L).

Use a tuple of 26 letter counts as the key (no sort). Total O(n * L); for large alphabets use a Counter turned into a frozenset of items or a sorted tuple of (char, count).

FOLLOW-UP 2 Case-insensitive, Unicode, or ignore spaces and punctuation.

Normalise first (casefold, strip non-letters, Unicode normalisation) and then compute the key. Say what counts as a letter.

FOLLOW-UP 3 The words are in many files.

Compute the key on each machine and hash-partition by key so every anagram group lands on one worker, which then groups locally (a MapReduce shuffle).

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
import unicodedata
from collections import defaultdict


# BASE QUESTION (optimal): Dictionary keyed by the sorted
# letters
def group_by_sorting(words):
    # anagrams share the same letters in sorted order
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())


# FOLLOW-UP 1: Make the key O(L) instead of O(L log L).
def group_by_counts(words):
    # key = 26 letter counts: O(L) per word instead of O(L
    # log L)
    groups = defaultdict(list)
    for w in words:
        counts = [0] * 26
        for ch in w:
            counts[ord(ch) - ord("a")] += 1
        groups[tuple(counts)].append(w)
    return list(groups.values())


# FOLLOW-UP 2: Case-insensitive, Unicode, or ignore spaces
# and punctuation.
def normalise(word):
    # case, accents, spaces and punctuation: decide what
    # counts
    text = unicodedata.normalize("NFKD", word).casefold()
    return "".join(ch for ch in text if ch.isalpha())


# FOLLOW-UP 2: Case-insensitive, Unicode, or ignore spaces
# and punctuation.
def group_normalised(words):
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(normalise(w)))].append(w)
    return list(groups.values())


# FOLLOW-UP 3: The words are in many files.
def shard_of(word, workers):
    # many files / machines: hash the key so one anagram
    # group
    # always lands on the same worker
    key = "".join(sorted(word))
    return sum(map(ord, key)) % workers


if __name__ == "__main__":
    words = ["eat", "tea", "tan", "ate", "nat", "bat"]
    want = [["eat", "tea", "ate"], ["tan", "nat"],
            ["bat"]]
    assert group_by_sorting(words) == want
    assert group_by_counts(words) == want
    assert group_by_sorting([]) == []
    assert group_by_sorting([""]) == [[""]]
    mixed = ["Listen", "Silent", "enlist", "a gentleman"]
    got = group_normalised(mixed)[0]
    assert got == ["Listen", "Silent",
                                        "enlist"]
    assert normalise("Café!") == "cafe"
    # anagrams always go to the same worker
    assert shard_of("eat", 8) == shard_of("tea", 8)
```

OFFICIAL PAGE TEXT

Given an array of strings, group together the words that are anagrams (e.g. gas, sag) of each other. The order of the groups or the words within the groups does not matter in the final result.

MEDIUM | IN APP LC 198 | Exponent | 33 of 33

★ TOP 75: rank 74 in the Medium and Hard priority list

★ Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

ASKED (ANY CO.) ~2 MONTHS AGO | LAST 90 DAYS

STUDY: TYPE | ALL TIME 57.2 | 30 DAYS 0 | PRIORITY 50/224

ABOUT THIS QUESTION

Topic: Dynamic Programming and State-Space Optimization

Optimal answer from: your app's interview approach (LeetCode #198)

LeetCode match: LC 198 House Robber (inferred from short statement)

Last asked (any company): 2 months ago, about 2026-08 (Exponent shows only a rough age)

Original: https://www.aced.io/questions/4646/house-robber

QUESTION

Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.

APPROACH

dp: rob = skip_prev + x, skip = max(rob_prev, skip_prev); O(n) time, O(1) space.

HOW TO THINK ABOUT IT

At each element you either take it (then the previous one is off limits) or skip it. Only two numbers matter from the past: the best total if you took the previous element and the best if you did not. Update them together and the whole array is one pass with no table.

Basis: LeetCode 198, House Robber: pick elements with no two adjacent to maximise the sum.

BRUTE FORCE

BRUTE FORCE · PSEUDO-CODE

```
def best(i):
    if i >= len(nums):
        return 0
    skip = best(i + 1)
    take = nums[i] + best(i + 2)
    return max(skip, take)
```

TIME	SPACE
O(2^n)	O(n)

WHY IT IS SLOW

Two calls per index and the same suffix is solved repeatedly: O(2^n). Keeping only the best total with and without the previous element is O(n) time and O(1) space.

OPTIMAL SOLUTION · Two running values (take, skip)

TIME	SPACE
O(n)	O(1)

Code: see FULL INTERVIEW CODE below (the optimal solution is its main function).

FOLLOW-UP 1 The houses form a circle (LeetCode 213).

The first and last are adjacent, so you cannot take both: the answer is the better of 'ignore the first house' and 'ignore the last house', each solved with the linear version.

FOLLOW-UP 2 Return which houses to rob.

Keep the table dp[i] = best from i onward (filled from the right) and walk forward: take house i when nums[i] + dp[i + 2] equals dp[i], then jump two; otherwise move one.

FOLLOW-UP 3 Robbing a house also alerts the next k houses.

The recurrence becomes dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1]); the O(1) space trick needs a window of k + 1 values (a deque), or just keep the O(n) table.

FULL INTERVIEW CODE · base question + every follow-up

One program for the whole question: the base answer and every follow-up above, built from small helper functions, with a comment above each step. Type it top to bottom in the interview; the demo at the bottom runs it.

```
# BASE QUESTION (optimal): Two running values (take, skip)
def rob_line(nums):
    # take = best sum if we take this house; skip = if we
    # do not
    take = skip = 0
    for x in nums:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)


# FOLLOW-UP 1: The houses form a circle (LeetCode 213).
def rob_circle(nums):
    # the first and last houses touch: ignore one of them
    if len(nums) == 1:
        return nums[0]
    return max(rob_line(nums[1:]), rob_line(nums[:-1]))


# FOLLOW-UP 2: Return which houses to rob.
def rob_with_choice(nums):
    # dp[i] = best from house i to the end (filled from
    # the right)
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + 2])
    # walk forward: take house i when it explains dp[i]
    houses, i = [], 0
    while i < n:
        if nums[i] > 0 and nums[i] + dp[i + 2] == dp[i]:
            houses.append(i)
            i += 2
        else:
            i += 1
    return dp[0], houses


# FOLLOW-UP 3: Robbing a house also alerts the next k
# houses.
def rob_alarm(nums, k):
    # robbing a house alerts the next k houses
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = max(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]


if __name__ == "__main__":
    assert rob_line([1, 2, 3, 1]) == 4
```

```
assert rob_line([2, 7, 9, 3, 1]) == 12
assert rob_line([]) == 0
assert rob_circle([2, 3, 2]) == 3
assert rob_circle([1, 2, 3, 1]) == 4
assert rob_circle([5]) == 5
total, houses = rob_with_choice([2, 7, 9, 3, 1])
assert total == 12 and houses == [0, 2, 4]
assert rob_alarm([2, 7, 9, 3, 1], 1) == 12
assert rob_alarm([5, 1, 1, 5, 1, 1, 5], 2) == 15
```

OFFICIAL PAGE TEXT

You are given an array representing the value in the houses e.g. [100, 20, 40, 70, 80]. Return 100 + 40 + 80 = 220